

INTERNATIONAL COLLEGIATE PROGRAMMING CONTEST
ACM-ICPC REFERENCE MANUAL

Team Notebook

Team:
HSG-sicula

JULY 19, 2026

Table of Contents

1	Environment Setup	2
1.1	Checklist & Code::Blocks Setup	2
1.2	Template & Fast I/O	3
1.3	Optimization & Diagnostic Pragmas	3
1.4	Custom Fast I/O Extensions	3
2	Data Structures	3
2.1	Chtholly Tree (Old Driver Tree)	3
2.2	Disjoint Set Union	3
2.3	DSU with Rollback	3
2.4	Fenwick Tree with lower/upper bound	4
2.5	Mo's Algorithm	4
2.6	Sparse Table RMQ	4
2.7	Segment Tree 2N (Iterative)	4
2.8	Segment Tree with Lazy Propagation	4
2.9	Persistent Segment Tree (Pointer Implementation)	5
2.10	Walk on Segment Tree	5
2.11	Treap (Randomized Binary Search Tree)	5
2.12	Implicit Treap	5
2.13	Wavelet Tree	6
3	Graphs and Trees	6
3.1	2-SAT Solver	6
3.2	Articulation Points & Bridges	6
3.3	Centroid Decomposition	6
3.4	Dijkstra Algorithm	7
3.5	Dinic Algorithm	7
3.6	Min Cost Max Flow (MCMF)	7
3.7	Flow with Lower Bound Setup Guide	8
3.8	Heavy-Light Decomposition (HLD)	8
3.9	Hopcroft-Karp Algorithm	8
3.10	Kruskal Reconstruction Tree (KRT)	9
3.11	LCA with Binary Lifting	9
3.12	LCA in O(1) with RMQ	9
3.13	Boruvka's Algorithm	10
3.14	Kruskal's Algorithm	10
3.15	Prim's Algorithm	10
3.16	Kosaraju's Algorithm	10
3.17	Tarjan Algorithm	11
3.18	Virtual Tree (Auxiliary Tree)	11
4	Math	11
4.1	Chinese Remainder Theorem (CRT)	11
4.2	Euler's Totient Function (Phi)	11

4.3	Extended Euclidean	12
4.4	FFT & NTT (Fast Fourier Transform & Number Theoretic Transform)	12
4.5	Matrix Exponentiation Template	12
4.6	Miller-Rabin Primality Test & Pollard's Rho	13
4.7	Fast Binary GCD (Stein's Algorithm)	13
4.8	Tonelli-Shanks Algorithm	13
5	Strings	13
5.1	Aho-Corasick Automaton	13
5.2	KMP (Knuth-Morris-Pratt) Algorithm	14
5.3	Manacher's Algorithm	14
5.4	Palindrome Tree (Eertree)	14
5.5	String Double Hashing	15
5.6	String Single Hashing	15
5.7	Suffix Array	15
5.8	Trie	15
5.9	Z-Algorithm	15
6	BitManipulation	16
6.1	Binary Trie (Bit Trie)	16
6.2	Bitwise Hacks	16
6.3	Linear Basis (XOR Basis)	16
7	DPs	16
7.1	Aliens Trick (WQS Binary Search)	16
7.2	Divide & Conquer Optimization	17
7.3	Knuth Optimization	17
8	Geometry	17
8.1	Convex Hull (Andrew's Monotone Chain)	17
8.2	Geometry Templates (Integer & Double)	17
8.3	Incenter & Circumcenter of a Triangle	18
8.4	Li Chao Tree	18
8.5	LineContainer (CHT)	18
8.6	Polygon Area (Shoelace Formula)	19
8.7	Smallest Enclosing Circle (Welzl's Algorithm)	19
9	Misc	19
9.1	Custom Hash for hashmap & hashset	19
9.2	Hill Climbing (Heuristics)	19
9.3	Simulated Annealing (Heuristics)	19
9.4	Policy-Based Data Structures (PBDS)	20
9.5	Stress Tester	20
10	Classic	20
10.1	Closest Pair of Points	20
11	Formulas	20
11.1	Advanced Graph Conjectures & Theorems	20
11.2	Essential Mathematics Formulas	21

11.3	Combinatorics & Modular Precomputations	21
11.4	Catalan Numbers	21
11.5	Derangements (Permutations with no fixed points)	21
11.6	Stirling Numbers of the Second Kind	21
11.7	Lucas' Theorem	21
11.8	Graph Theory Counting	21
11.9	Geometry Theorems	21
11.10	Essential Graph and Number Theory Theorems	22

1 Environment Setup

1.1 Checklist & Code::Blocks Setup

Contest preparation, environment validation, and Code::Blocks Abbreviation snippets configuration.

- Bring identities, credentials, pens, and printed Team Notebook.
- Verify system time, keyboard layout, and workspace comfort.
- Configure Code::Blocks:
Settings -> Compiler -> Compiler Settings -> Flag "-std=c++17" (or "-std=c++14").
- Add compiler flags: -O3, -Wall, -Wextra, and -Wshadow.
- Enable auto-save:
Settings -> Environment -> Autosave -> Check every 1 minute.
- Configure Abbreviations in Settings -> Editor -> Abbreviations to add quick snippets for freopen, standard template, and #define macros, then use Ctrl + J to expand them.
- Write a stress tester script to automatically generate test cases, compare output files using diff -w, and look for counter-examples.

Essential Code::Blocks Shortcuts:

- Ctrl + Shift + c: Comment selected block.
- Ctrl + Shift + x: Uncomment selected block.
- Ctrl + F9: Compile project/file.
- Ctrl + F10: Run compiled executable.
- F9: Compile and run instantly.
- Ctrl + Space: Open autocomplete suggestions.

- Ctrl + Shift + Space: Show function parameter tips.
- Alt + G: Jump to declaration or definition.
- Alt + Up/Down Arrow: Move selected lines up or down.
- Ctrl + E: Select next occurrence of the same keyword.
- Ctrl + Click: Multi-edit at custom positions.
- Ctrl + D: Duplicate selected text or current line.
- Ctrl + T: Swap current line with the line above it.

1.2 Template & Fast I/O

Optimized standard input/output streams execution.

```
ios_base::sync_with_stdio(0); cin.tie(0);
```

Standard configuration template for competitive programming.

```
#include <bits/stdc++.h>
using namespace std;

void solve() {

}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int t = 1;
    cin >> t;
    while (t--) {
        solve();
    }
    return 0;
}
```

1.3 Optimization & Diagnostic Pragmas

Enables advanced compiler optimizations, loop unrolling, and vectorization targeted for modern architectures, while silencing diagnostic warnings on older judge systems.

```
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```

Place these optimization lines at the absolute top of your source file, before any include statements. The `avx2` enables 256-bit SIMD vectorization for speedups in tight loops.

```
#pragma GCC diagnostic ignored "-Wpragmas"
#pragma GCC diagnostic ignored "-Wunused-result"
#pragma GCC diagnostic ignored "-Wunknown-pragmas"
```

The diagnostic directives prevent compilation warnings or failures on older platforms that do not support modern targets, and silence warnings for ignored return values (e.g., from `scanf` or `freopen`).

1.4 Custom Fast I/O Extensions

Platform-independent low-level fast I/O implementation using unlocked character reading for extreme execution speed.

```
#ifdef _WIN32
#define GETCHAR _getchar_nolock
#else
#define GETCHAR getchar_unlocked
#endif

inline int readInt() {
    int x = 0;
    char ch = GETCHAR();
    while (ch < '0' || ch > '9') {
        if (ch == EOF) return -1;
        ch = GETCHAR();
    }
    while (ch >= '0' && ch <= '9') {
        x = (x << 3) + (x << 1) + (ch - '0');
        ch = GETCHAR();
    }
    return x;
}

inline long long readLL() {
    long long x = 0;
    char ch = GETCHAR();
    while (ch < '0' || ch > '9') {
        if (ch == EOF) return -1;
        ch = GETCHAR();
    }
    while (ch >= '0' && ch <= '9') {
        x = (x << 3) + (x << 1) + (ch - '0');
        ch = GETCHAR();
    }
    return x;
}
```

2 Data Structures

2.1 Chtholly Tree (Old Driver Tree)

An efficient data structure for processing range assignment queries on arrays using `std::set`. It maintains segments of identical values as elements. Average time complexity is $O(M \log \log N)$ for random inputs where segment merge/assign operations occur frequently.

```
struct Node {
    int l, r;
    mutable long long val;
    Node(int l, int r = -1, long long val = 0) : l(l), r(r), val(val) {}
    bool operator<(const Node& o) const { return l < o.l; }
};

struct ChthollyTree {
    set<Node> odt;

    void init(const vector<long long>& a) {
        odt.clear();
        for (int i = 1; i < a.size(); i++) {
            odt.insert(Node(i, i, a[i]));
        }
    }

    // Splits the segment containing pos into [l, pos-1] and [pos, r]
```

```
// Returns an iterator pointing to the newly created segment
// starting at pos
auto split(int pos) {
    auto it = odt.lower_bound(Node(pos));
    if (it != odt.end() && it->l == pos) return it;
    --it;
    if (it->r < pos) return odt.end();
    int l = it->l, r = it->r;
    long long v = it->val;
    odt.erase(it);
    odt.insert(Node(l, pos - 1, v));
    return odt.insert(Node(pos, r, v)).first;
}

// Assigns a single uniform value to the entire range [l, r]
void assign(int l, int r, long long val) {
    auto itr = split(r + 1);
    auto itl = split(l);
    odt.erase(itl, itr);
    odt.insert(Node(l, r, val));
}

// Example range query operation (add value to range)
void add_range(int l, int r, long long val) {
    auto itr = split(r + 1);
    auto itl = split(l);
    for (; itl != itr; ++itl) {
        itl->val += val;
    }
}

};
```

Always call `split(r + 1)` before `split(l)` to protect the lower bound iterator validity from being invalidated during structural modification steps.

2.2 Disjoint Set Union

Disjoint Set Union (DSU) with path compression and size-based merging optimizations in $O(\alpha(N))$.

```
struct DSU {
    vector<int> parent, sz;
    DSU(int n) {
        parent.resize(n + 1); sz.assign(n + 1, 1);
        iota(parent.begin(), parent.end(), 0);
    }
    int find_set(int v) {
        if (v == parent[v]) return v;
        return parent[v] = find_set(parent[v]);
    }
    bool union_sets(int a, int b) {
        a = find_set(a); b = find_set(b);
        if (a != b) {
            if (sz[a] < sz[b]) swap(a, b);
            parent[b] = a; sz[a] += sz[b];
            return true;
        }
        return false;
    }
};
```

2.3 DSU with Rollback

Disjoint Set Union with history logging to undo merge operations. Path compression is disabled to preserve tree structural history, maintaining operations in $O(\log N)$.

```

struct DSU_Rollback {
    vector<int> parent, sz;
    vector<pair<int, int>> history;
    DSU_Rollback(int n) {
        parent.resize(n + 1); sz.assign(n + 1, 1);
        iota(parent.begin(), parent.end(), 0);
    }
    int find_set(int v) {
        while (v != parent[v]) v = parent[v];
        return v;
    }
    int get_time() { return history.size(); }
    bool union_sets(int a, int b) {
        a = find_set(a); b = find_set(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        history.push_back({b, parent[b]});
        history.push_back({a, sz[a]});
        parent[b] = a; sz[a] += sz[b];
        return true;
    }
    void rollback(int t) {
        while (history.size() > t) {
            auto [u, val] = history.back(); history.pop_back();
            if (u < 0) sz[u] = val;
            else parent[u] = val;
        }
    }
};

```

Call `int t = dsu.get_time()` to save the state checkpoint before processing speculative edges. Pass `t` to `dsu.rollback(t)` to revert the components back to that structural checkpoint.

2.4 Fenwick Tree with lower/upper bound

Finds the smallest index with a given prefix sum target in $O(\log N)$ using binary lifting. `lower_bound` finds the first index where the prefix sum is $\geq$ target. `upper_bound` finds the first index where the prefix sum is $>$ target.

```

struct BIT {
    int n; vector<int> tree;
    BIT(int n) : n(n), tree(n + 1, 0) {}
    void update(int i, int delta) {
        for (; i <= n; i += i & -i) tree[i] += delta;
    }
    int lower_bound(int sum) {
        int idx = 0;
        for (int i = 1 << __lg(n); i > 0; i >>= 1) {
            if (idx + i <= n && tree[idx + i] < sum) {
                idx += i; sum -= tree[idx];
            }
        }
        return idx + 1;
    }
    int upper_bound(int sum) {
        int idx = 0;
        for (int i = 1 << __lg(n); i > 0; i >>= 1) {
            if (idx + i <= n && tree[idx + i] <= sum) {
                idx += i; sum -= tree[idx];
            }
        }
        return idx + 1;
    }
};

```

2.5 Mo's Algorithm

An offline query processing structure that groups and sorts queries into square-root-sized blocks to process range queries in $O((N + Q)\sqrt{N})$ total time.

```

const int BLOCK_SIZE = 450;

struct Query {
    int l, r, id;
    bool operator<(const Query& other) const {
        int block_this = l / BLOCK_SIZE;
        int block_other = other.l / BLOCK_SIZE;
        if (block_this != block_other) return block_this <
            block_other;
        return (block_this & 1) ? (r < other.r) : (r > other.r);
    }
};

long long current_answer = 0;
vector<int> cnt;

void add(int idx, const vector<int>& a) {
}

void remove(int idx, const vector<int>& a) {
}

vector<long long> solve_mo(int n, const vector<int>& a, vector<
    Query>& queries) {
    int q = queries.size();
    sort(queries.begin(), queries.end());

    cnt.assign(1000005, 0);
    vector<long long> answers(q);
    int cur_l = 0, cur_r = -1;

    for (const auto& query : queries) {
        while (cur_l > query.l) { cur_l--; add(cur_l, a); }
        while (cur_r < query.r) { cur_r++; add(cur_r, a); }
        while (cur_l < query.l) { remove(cur_l, a); cur_l++; }
        while (cur_r > query.r) { remove(cur_r, a); cur_r--; }
        answers[query.id] = current_answer;
    }
    return answers;
}

```

2.6 Sparse Table RMQ

Static Range Minimum Query (RMQ) initialization in $O(N \log N)$ and query execution in $O(1)$.

```

struct SparseTable {
    int n, K; vector<vector<int>> st;
    SparseTable(const vector<int>& a) {
        n = a.size(); K = __lg(n);
        st.assign(K + 1, vector<int>(n));
        for (int i = 0; i < n; i++) st[0][i] = a[i];
        for (int j = 1; j <= K; j++) {
            for (int i = 0; i + (1 << j) <= n; i++) {
                st[j][i] = min(st[j - 1][i], st[j - 1][i + (1 << (j
                    - 1))]);
            }
        }
    }
    int query(int l, int r) {
        int j = __lg(r - l + 1);
        return min(st[j][l], st[j][r - (1 << j) + 1]);
    }
};

```

```
};
```

2.7 Segment Tree 2N (Iterative)

Point update and range query in $O(\log N)$ using a non-recursive $2N$ memory array. This implementation uses 0-indexed positioning for elements $0 \leq i < n$, but operates efficiently on the internal array. `update` modifies a specific element, and `query` computes the aggregate sum over the range $[l, r]$.

```

struct SegTree {
    int n; vector<int> tree;
    SegTree(int n) : n(n), tree(2 * n, 0) {}
    void update(int i, int value) {
        for (tree[i += n] = value; i > 1; i >>= 1) {
            tree[i >> 1] = tree[i] + tree[i ^ 1];
        }
    }
    int query(int l, int r) {
        int res = 0;
        for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
            if (l & 1) res += tree[l++];
            if (r & 1) res += tree[--r];
        }
        return res;
    }
};

```

To adapt this structure for a 1-indexed array where elements range from $1 \leq i \leq n$, change the initialization size to `tree(2 * (n + 1), 0)`. When calling functions, simply pass the indices directly as `update(i, val)` and `query(l, r)` without manual shifts, as the internal bitwise operations automatically maintain the structural layout.

2.8 Segment Tree with Lazy Propagation

Range update and range query in $O(\log N)$.

```

struct SegTreeLazy {
    int n; vector<int> tree, lazy;
    SegTreeLazy(int n) : n(n), tree(4 * n, 0), lazy(4 * n, 0) {}
    void push(int node, int l, int r) {
        if (!lazy[node]) return;
        int mid = (l + r) >> 1;
        tree[node << 1] += (mid - l + 1) * lazy[node];
        lazy[node << 1] += lazy[node];
        tree[node << 1 | 1] += (r - mid) * lazy[node];
        lazy[node << 1 | 1] += lazy[node];
        lazy[node] = 0;
    }
    void update(int node, int l, int r, int ql, int qr, int val) {
        if (ql <= l && r <= qr) {
            tree[node] += (r - l + 1) * val;
            lazy[node] += val;
            return;
        }
        push(node, l, r);
        int mid = (l + r) >> 1;
        if (ql <= mid) update(node << 1, l, mid, ql, qr, val);
        if (qr > mid) update(node << 1 | 1, mid + 1, r, ql, qr, val);
    }
    int query(int node, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return tree[node];
        push(node, l, r);
    }
};

```

```

    int mid = (l + r) >> 1, res = 0;
    if (ql <= mid) res += query(node << 1, 1, mid, ql, qr);
    if (qr > mid) res += query(node << 1 | 1, mid + 1, r, ql, qr);
    return res;
}
};

```

2.9 Persistent Segment Tree (Pointer Implementation)

Maintains multiple historical versions of a Segment Tree using explicit pointers. Each point update creates at most $O(\log N)$ new nodes, preserving previous versions in $O(\log N)$ time and space.

```

struct Node {
    int val;
    Node *l, *r;
    Node(int val = 0) : val(val), l(nullptr), r(nullptr) {}
    Node(Node* original) {
        val = original->val;
        l = original->l;
        r = original->r;
    }
};

struct PersistentSegTree {
    int n;
    vector<Node*> roots;

    PersistentSegTree(int n) : n(n) {
        roots.push_back(build(1, n));
    }

    Node* build(int l, int r) {
        Node* curr = new Node(0);
        if (l == r) return curr;
        int mid = (l + r) >> 1;
        curr->l = build(l, mid);
        curr->r = build(mid + 1, r);
        return curr;
    }

    Node* update(Node* prev_node, int l, int r, int pos, int val) {
        Node* curr = new Node(prev_node);
        if (l == r) {
            curr->val += val;
            return curr;
        }
        int mid = (l + r) >> 1;
        if (pos <= mid) curr->l = update(prev_node->l, l, mid, pos, val);
        else curr->r = update(prev_node->r, mid + 1, r, pos, val);
        curr->val = curr->l->val + curr->r->val;
        return curr;
    }

    int query(Node* node, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return node->val;
        int mid = (l + r) >> 1, res = 0;
        if (ql <= mid) res += query(node->l, l, mid, ql, qr);
        if (qr > mid) res += query(node->r, mid + 1, r, ql, qr);
        return res;
    }

    void upgrade(int version, int pos, int val) {
        roots.push_back(update(roots[version], 1, n, pos, val));
    }
};

```

Initialize via PersistentSegTree pst(n). Call pst.upgrade(v, pos, val) to branch a new version from version v. Query data from version v via pst.query(pst.roots[v], 1, n, ql, qr).

2.10 Walk on Segment Tree

Finds the first position in an array that satisfies a given condition (e.g., finding the first index i such that $A[i] \geq X$) in $O(\log N)$ by descending the segment tree nodes directly.

```

// Returns the first index i in [ql, qr] where tree[node] >= x.
// Returns -1 if none exists.
int walk(int node, int l, int r, int ql, int qr, int x) {
    if (l > qr || r < ql || tree[node] < x) return -1;
    if (l == r) return l;

    int mid = (l + r) >> 1;
    int res = walk(node << 1, 1, mid, ql, qr, x);
    if (res != -1) return res;

    return walk(node << 1 | 1, mid + 1, r, ql, qr, x);
}

```

2.11 Treap (Randomized Binary Search Tree)

A balanced binary search tree combining BST and Heap properties, executing insertion, deletion, and splits in $O(\log N)$ expected time using randomized priorities.

```

mt19937 treap_rng(1337);

struct Node {
    int key, prior;
    int sz;
    Node *l, *r;
    Node(int key) : key(key), prior(treap_rng()), sz(1), l(nullptr), r(nullptr) {}
};

int get_sz(Node* t) { return t ? t->sz : 0; }
void upd_sz(Node* t) { if (t) t->sz = 1 + get_sz(t->l) + get_sz(t->r); }

void split(Node* t, int key, Node*& l, Node*& r) {
    if (!t) l = r = nullptr;
    else if (t->key <= key) {
        split(t->r, key, t, r);
        l = t; upd_sz(l);
    } else {
        split(t->l, key, l, t);
        r = t; upd_sz(r);
    }
}

void merge(Node*& t, Node* l, Node* r) {
    if (!l || !r) t = l ? l : r;
    else if (l->prior > r->prior) {
        merge(l->r, l->r, r);
        t = l; upd_sz(t);
    } else {
        merge(r->l, l, r->l);
        t = r; upd_sz(t);
    }
}

void insert(Node*& t, Node* it) {
    if (!t) t = it;
    else if (it->prior > t->prior) {
        split(t, it->key, it->l, it->r);
    }
}

```

```

    t = it; upd_sz(t);
} else {
    insert(t->key <= it->key ? t->r : t->l, it);
    upd_sz(t);
}

void erase(Node*& t, int key) {
    if (!t) return;
    if (t->key == key) {
        Node* th = t;
        merge(t, t->l, t->r);
        delete th;
    } else {
        erase(key < t->key ? t->l : t->r, key);
        upd_sz(t);
    }
}

```

2.12 Implicit Treap

An implicit balanced binary search tree that indexes elements by their array positions rather than explicit keys. It supports range operations, updates, and array splits/merges in $O(\log N)$ expected time.

```

mt19937 implicit_rng(1337);

struct Node {
    int val, prior;
    int sz;
    bool rev;
    Node *l, *r;
    Node(int val) : val(val), prior(implicit_rng()), sz(1), rev(false), l(nullptr), r(nullptr) {}
};

int get_sz(Node* t) { return t ? t->sz : 0; }
void upd_sz(Node* t) { if (t) t->sz = 1 + get_sz(t->l) + get_sz(t->r); }

void push(Node* t) {
    if (t && t->rev) {
        t->rev = false;
        swap(t->l, t->r);
        if (t->l) t->l->rev ^= true;
        if (t->r) t->r->rev ^= true;
    }
}

void split(Node* t, int add, Node*& l, Node*& r) {
    if (!t) { l = r = nullptr; return; }
    push(t);
    int curr_idx = add + get_sz(t->l);
    if (curr_idx < add) {
        split(t->r, curr_idx + 1, t->r, r);
        l = t; upd_sz(l);
    } else {
        split(t->l, add, l, t->l);
        r = t; upd_sz(r);
    }
}

void merge(Node*& t, Node* l, Node* r) {
    push(l); push(r);
    if (!l || !r) t = l ? l : r;
    else if (l->prior > r->prior) {
        merge(l->r, l->r, r);
        t = l; upd_sz(t);
    } else {
        merge(r->l, l, r->l);
        t = r; upd_sz(t);
    }
}

```

```

    t = r; upd_sz(t);
}

void reverse_range(Node*& t, int ql, int qr) {
    Node *l1, *r1, *l2, *r2;
    split(t, ql, l1, r1);
    split(r1, qr - ql + 1, l2, r2);
    if (l2) l2->rev ^= true;
    merge(r1, l2, r2);
    merge(t, l1, r1);
}

```

2.13 Wavelet Tree

A powerful data structure that supports generalized range queries (like finding the k -th smallest element or the number of elements in range $[l, r]$ less than or equal to k) in $O(\log(\max A_i))$ time.

```

struct WaveletTree {
    int lo, hi;
    WaveletTree *l, *r;
    vector<int> b;

    // Elements must be in range [lo, hi], sequence passes by
    // iterators
    template<typename Iter>
    WaveletTree(Iter from, Iter to, int lo, int hi) : lo(lo), hi(hi)
    {
        l(nullptr), r(nullptr) {
            if (from == to || lo == hi) return;
            int mid = (lo + hi) >> 1;
            auto f = [mid](int x) { return x <= mid; };
            b.reserve(to - from + 1);
            b.push_back(0);
            for (auto it = from; it != to; ++it) {
                b.push_back(b.back() + f(*it));
            }
            auto pivot = stable_partition(from, to, f);
            l = new WaveletTree(from, pivot, lo, mid);
            r = new WaveletTree(pivot, to, mid + 1, hi);
        }

        // Returns the k-th smallest element in range [l, r] (1-indexed
        // k)
        int kth(int l, int r, int k) {
            if (lo == hi) return lo;
            int in_left = b[r] - b[l - 1];
            int lb = b[l - 1], rb = b[r];
            if (k <= in_left) return l->kth(lb + 1, rb, k);
            return r->kth(lb + 1, rb, k - in_left);
        }

        // Returns count of elements in range [l, r] less than or equal
        // to k
        int count_le(int l, int r, int k) {
            if (l > r || k < lo) return 0;
            if (hi <= k) return r - l + 1;
            int lb = b[l - 1], rb = b[r];
            return l->count_le(lb + 1, rb, k) + r->count_le(lb + 1, rb, k - rb, k);
        }

        ~WaveletTree() {
            delete l;
            delete r;
        }
};

```

Initialize via `WaveletTree T(a.begin(), a.end(), min_val, max_val)`. Query

operations like `T.kth(l, r, k)` and `T.count_le(l, r, k)` expect 1-indexed range boundaries.

3 Graphs and Trees

3.1 2-SAT Solver

Solves the 2-Satisfiability problem for a set of clauses $(u \vee v)$ in $O(V + E)$ using Tarjan's Strongly Connected Components algorithm.

```

struct TwoSat {
    int n;
    vector<vector<int>> adj;
    vector<int> tin, low, scc_id;
    vector<bool> in_stack, answer;
    stack<int> st;
    int timer, scc_count;

    TwoSat(int n) : n(n), adj(2 * n + 1), tin(2 * n + 1, -1), low(2 * n + 1, -1),
        scc_id(2 * n + 1, -1), in_stack(2 * n + 1, false), answer(n + 1, false),
        timer(0), scc_count(0) {}

    // Var input uses 1-indexed values. Use negative sign for
    // negation (e.g., -u)
    void add_clause(int u, int v) {
        int not_u = (u > 0) ? u + n : -u;
        int act_u = (u > 0) ? u : -u + n;
        int not_v = (v > 0) ? v + n : -v;
        int act_v = (v > 0) ? v : -v + n;
        adj[not_u].push_back(act_v);
        adj[not_v].push_back(act_u);
    }

    void dfs(int u) {
        tin[u] = low[u] = ++timer;
        st.push(u);
        in_stack[u] = true;
        for (int v : adj[u]) {
            if (tin[v] == -1) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (in_stack[v]) {
                low[u] = min(low[u], tin[v]);
            }
        }
        if (low[u] == tin[u]) {
            scc_count++;
            while (true) {
                int v = st.top(); st.pop();
                in_stack[v] = false;
                scc_id[v] = scc_count;
                if (u == v) break;
            }
        }
    }

    bool solve() {
        for (int i = 1; i <= 2 * n; i++) {
            if (tin[i] == -1) dfs(i);
        }
        for (int i = 1; i <= n; i++) {
            if (scc_id[i] == scc_id[i + n]) return false;
            answer[i] = scc_id[i] < scc_id[i + n];
        }
        return true;
    }
};

```

Initialize via `TwoSat solver(n)`. Pass your clauses using 1-indexed integers into `solver.add_clause(u, v)`. Call `solver.solve()` which returns a boolean indicating satisfiability. The true/false configuration for variable i is populated directly inside `solver.answer[i]`.

3.2 Articulation Points & Bridges

Finds all articulation points (cut vertices) and bridges in an undirected graph in $O(V + E)$ using a single DFS traversal (Tarjan/Hopcroft-Tarjan algorithm).

```

struct CutVerticesBridges {
    int n, timer;
    vector<vector<int>> adj;
    vector<int> tin, low;
    vector<bool> is_cut;
    vector<pair<int, int>> bridges;

    CutVerticesBridges(int n, const vector<vector<int>>& adj) :
        n(n), adj(adj), timer(0),
        tin(n + 1, -1), low(n + 1, -1), is_cut(n + 1, false) {}

    void dfs(int u, int p = -1) {
        tin[u] = low[u] = ++timer;
        int children = 0;

        for (int v : adj[u]) {
            if (v == p) continue;
            if (tin[v] != -1) {
                low[u] = min(low[u], tin[v]);
            } else {
                dfs(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] >= tin[u] && p != -1) {
                    is_cut[u] = true;
                }
                if (low[v] > tin[u]) {
                    bridges.push_back({min(u, v), max(u, v)});
                }
                children++;
            }
        }
        if (p == -1 && children > 1) {
            is_cut[u] = true;
        }
    }

    void run() {
        for (int i = 1; i <= n; i++) {
            if (tin[i] == -1) dfs(i);
        }
    }
};

```

3.3 Centroid Decomposition

Divides a tree into a tree of centroids in $O(N \log N)$ total time. It is highly effective for solving tree path queries by ensuring that the maximum component size is halved at each step, bounding the maximum centroid tree depth at $O(\log N)$.

```

struct CentroidDecomposition {
    int n;
    vector<vector<int>> adj;
    vector<int> sz, parent;
    vector<bool> removed;
};

```

```

CentroidDecomposition(int n, const vector<vector<int>>&
tree_adj) :
n(n), adj(tree_adj), sz(n + 1), parent(n + 1, 0), removed(n
+ 1, false) {}

int get_sz(int u, int p) {
sz[u] = 1;
for (int v : adj[u]) {
if (v != p && !removed[v]) {
sz[u] += get_sz(v, u);
}
}
return sz[u];
}

int find_centroid(int u, int p, int comp_sz) {
for (int v : adj[u]) {
if (v != p && !removed[v] && sz[v] > comp_sz / 2) {
return find_centroid(v, u, comp_sz);
}
}
return u;
}

int build_tree(int u) {
int comp_sz = get_sz(u, 0);
int centroid = find_centroid(u, 0, comp_sz);
removed[centroid] = true;

for (int v : adj[centroid]) {
if (!removed[v]) {
int child_centroid = build_tree(v);
parent[child_centroid] = centroid;
}
}
return centroid;
}

void init(int start_node = 1) {
build_tree(start_node);
}
};

```

Initialize via `CentroidDecomposition cd(n, adj)` and call `cd.init()`. The generated structural tree layout is stored in `parent`, allowing distance or value statistics to be updated upwards through centroid ancestors.

3.4 Dijkstra Algorithm

Shortest path algorithm from a single source vertex to all other vertices. The adjacency list with `priority_queue` version runs in $O(E \log V)$ and is optimal for sparse graphs. The adjacency matrix version runs in $O(V^2)$ and is optimal for dense graphs ($E \approx V^2$).

```

const long long INF = 1e18;
struct Edge { int to, weight; };

vector<long long> dijkstra(int source, int n, const vector<vector<
Edge>>& adj) {
vector<long long> dist(n + 1, INF);
priority_queue<pair<long long, int>, vector<pair<long long, int>
>>, greater<pair<long long, int>>> pq;

dist[source] = 0;
pq.push({0, source});

while (!pq.empty()) {

```

```

auto [d, u] = pq.top(); pq.pop();
if (d > dist[u]) continue;

for (const auto& edge : adj[u]) {
if (dist[u] + edge.weight < dist[edge.to]) {
dist[edge.to] = dist[u] + edge.weight;
pq.push({dist[edge.to], edge.to});
}
}

return dist;
}

vector<long long> dijkstra_dense(int source, int n, const vector<
vector<long long>>& adj) {
vector<long long> dist(n + 1, INF);
vector<bool> visited(n + 1, false);

dist[source] = 0;
for (int i = 1; i <= n; i++) {
int u = -1;
for (int v = 1; v <= n; v++) {
if (!visited[v] && (u == -1 || dist[v] < dist[u])) {
u = v;
}
}

if (dist[u] == INF) break;
visited[u] = true;

for (int v = 1; v <= n; v++) {
if (adj[u][v] != INF && dist[u] + adj[u][v] < dist[v]) {
dist[v] = dist[u] + adj[u][v];
}
}
}

return dist;
}

```

3.5 Dinic Algorithm

Max flow algorithm running in $O(V^2E)$ in the worst case, but significantly faster in practice, particularly on unit networks ($O(E\sqrt{V})$).

```

const long long INF = 1e18;

struct Dinic {
struct Edge { int to; long long flow, cap; int rev; };
int n, source, sink;
vector<int> level, ptr;
vector<vector<Edge>> adj;

Dinic(int n, int source, int sink) : n(n), source(source), sink
(sink), level(n + 1), ptr(n + 1), adj(n + 1) {}

void add_edge(int from, int to, long long cap) {
adj[from].push_back({to, 0, cap, (int)adj[to].size()});
adj[to].push_back({from, 0, 0, (int)adj[from].size() - 1});
}

bool bfs() {
fill(level.begin(), level.end(), -1);
level[source] = 0;
queue<int> q; q.push(source);
while (!q.empty()) {
int u = q.front(); q.pop();
for (auto& edge : adj[u]) {
if (edge.cap - edge.flow > 0 && level[edge.to] ==
-1) {
level[edge.to] = level[u] + 1;

```

```

q.push(edge.to);
}
}

return level[sink] != -1;
}

long long dfs(int u, long long pushed) {
if (pushed == 0 || u == sink) return pushed;
for (int& cid = ptr[u]; cid < adj[u].size(); ++cid) {
auto& edge = adj[u][cid];
int tr = edge.to;
if (level[u] + 1 != level[tr] || edge.cap - edge.flow
== 0) continue;
long long tr_pushed = dfs(tr, min(pushed, edge.cap -
edge.flow));
if (tr_pushed == 0) continue;
edge.flow += tr_pushed;
adj[tr][edge.rev].flow -= tr_pushed;
return tr_pushed;
}
return 0;
}

long long max_flow() {
long long flow = 0;
while (bfs()) {
fill(ptr.begin(), ptr.end(), 0);
while (long long pushed = dfs(source, INF)) {
flow += pushed;
}
}
return flow;
}
};

```

3.6 Min Cost Max Flow (MCMF)

Finds the maximum flow with the minimum total cost in $O(F \cdot E \log V)$ using SPFA (or Dijkstra with potentials) for networks without negative cycles.

```

const long long INF = 1e18;

struct MCMF {
struct Edge { int to; long long cap, flow, cost; int rev; };
int n, source, sink;
vector<vector<Edge>> adj;
vector<long long> dist;
vector<int> parent, parent_edge;
vector<bool> in_queue;

MCMF(int n, int source, int sink) : n(n), source(source), sink(
sink), adj(n + 1), dist(n + 1), parent(n + 1), parent_edge(n +
1), in_queue(n + 1) {}

void add_edge(int from, int to, long long cap, long long cost)
{
adj[from].push_back({to, cap, 0, cost, (int)adj[to].size()
});
adj[to].push_back({from, 0, 0, -cost, (int)adj[from].size()
- 1});
}

bool spfa() {
fill(dist.begin(), dist.end(), INF);
fill(in_queue.begin(), in_queue.end(), false);
queue<int> q;

dist[source] = 0;
q.push(source);
in_queue[source] = true;

```



```

while (!q.empty()) {
    int u = q.front(); q.pop();
    in_queue[u] = false;

    for (int i = 0; i < adj[u].size(); i++) {
        auto& edge = adj[u][i];
        if (edge.cap - edge.flow > 0 && dist[u] + edge.cost
            < dist[edge.to]) {
            dist[edge.to] = dist[u] + edge.cost;
            parent[edge.to] = u;
            parent_edge[edge.to] = i;
            if (!in_queue[edge.to]) {
                q.push(edge.to);
                in_queue[edge.to] = true;
            }
        }
    }
}

return dist[sink] != INF;
}

pair<long long, long long> solve() {
    long long max_flow = 0, min_cost = 0;
    while (spfa()) {
        long long pushed = INF;
        for (int v = sink; v != source; v = parent[v]) {
            int u = parent[v];
            int idx = parent_edge[v];
            pushed = min(pushed, adj[u][idx].cap - adj[u][idx].
                flow);
        }
        for (int v = sink; v != source; v = parent[v]) {
            int u = parent[v];
            int idx = parent_edge[v];
            adj[u][idx].flow += pushed;
            adj[v][adj[u][idx].rev].flow -= pushed;
            min_cost += pushed * adj[u][idx].cost;
        }
        max_flow += pushed;
    }
    return {max_flow, min_cost};
}
};

```

Initialize via MCMF solver(n , source, sink). Push paths via solver.add_edge(u , v , cap, cost) and call solver.solve() to retrieve the {max_flow, min_cost} pair coordinates.

3.7 Flow with Lower Bound Setup Guide

Graph transformation steps to solve network flows with a lower bound constraint $[L(u, v), C(u, v)]$ on edges.

Graph Transformation Steps:

1. **Modify Edges:** Replace each original edge (u, v) with capacity $C - L$.
2. **Track Vertex Demand:** Maintain an array delta[u] initialized to 0. For each original edge (u, v) with lower bound L :

$$\text{delta}[u] \leftarrow \text{delta}[u] - L, \quad \text{delta}[v] \leftarrow \text{delta}[v] + L$$

3. **Add Super Source (S') & Super Sink (T'):** For every vertex u :

- If $\text{delta}[u] > 0$: Add edge (S', u) with capacity $\text{delta}[u]$.
- If $\text{delta}[u] < 0$: Add edge (u, T') with capacity $-\text{delta}[u]$.

Feasible Flow (No Source/Sink): Run Max Flow from S' to T' .

The flow is feasible if and only if the max_flow equals the sum of all positive $\text{delta}[u]$. Real flow on edge $(u, v) = L(u, v) + \text{current_flow}(u, v)$.

Max/Min Flow with Source (S) and Sink (T):

1. Add a feedback edge (T, S) with capacity ∞ and lower bound 0 to make the network circulatory.
2. Run Max Flow from S' to T' . Check feasibility (must saturate all S' edges).
3. Store the temporary flow value:
initial_flow = flow on edge (T, S) .
4. Remove the feedback edge (T, S) from the graph.
5. **For Maximum Flow:** Run Max Flow from S to T on the residual graph.

$$\text{Max Flow} = \text{initial_flow} + \text{new_flow}(S, T)$$

6. **For Minimum Flow:** Run Max Flow from T to S on the residual graph to return excess flow.

$$\text{Min Flow} = \text{initial_flow} - \text{new_flow}(T, S)$$

3.8 Heavy-Light Decomposition (HLD)

Decomposes a tree into paths where each vertex belongs to exactly one heavy path. Allows range queries and updates on paths in $O(\log^2 N)$ when paired with a Segment Tree.

```

struct HLD {
    int n, timer;
    vector<vector<int>>> adj;
    vector<int> parent, depth, heavy, head, pos;

    HLD(int n, const vector<vector<int>>& tree_adj) :
        n(n), timer(0), adj(tree_adj),
        parent(n + 1), depth(n + 1), heavy(n + 1, -1), head(n + 1),
        pos(n + 1) {}

    int dfs_sz(int u, int p, int d) {
        int size = 1, max_c_size = 0;
        parent[u] = p; depth[u] = d;
        for (int v : adj[u]) {
            if (v != p) {
                int c_size = dfs_sz(v, u, d + 1);
                size += c_size;
                if (c_size > max_c_size) {
                    max_c_size = c_size;
                    heavy[u] = v;
                }
            }
        }
    }
};

```

```

}
return size;
}

void decompose(int u, int h) {
    head[u] = h; pos[u] = ++timer;
    if (heavy[u] != -1) {
        decompose(heavy[u], h);
    }
    for (int v : adj[u]) {
        if (v != parent[u] && v != heavy[u]) {
            decompose(v, v);
        }
    }
}

void init(int root = 1) {
    dfs_sz(root, 0, 0);
    decompose(root, root);
}

template<typename SegTree>
int query_path(int u, int v, SegTree& seg) {
    int res = 0; // Set to identity element (e.g., -INF for max, 0 for sum)
    for (; head[u] != head[v]; v = parent[head[v]]) {
        if (depth[head[u]] > depth[head[v]]) swap(u, v);
        res += seg.query(1, 1, n, pos[head[v]], pos[v]); //
        // Modify operation accordingly
    }
    if (depth[u] > depth[v]) swap(u, v);
    res += seg.query(1, 1, n, pos[u], pos[v]); // Node values
    // query
    // For edge values query, use: pos[heavy_child_of_u]
    // instead of pos[u]
    return res;
}

template<typename SegTree>
void update_path(int u, int v, int val, SegTree& seg) {
    for (; head[u] != head[v]; v = parent[head[v]]) {
        if (depth[head[u]] > depth[head[v]]) swap(u, v);
        seg.update(1, 1, n, pos[head[v]], pos[v], val);
    }
    if (depth[u] > depth[v]) swap(u, v);
    seg.update(1, 1, n, pos[u], pos[v], val);
}
};

```

Initialize via HLD hld(n , adj) and call hld.init(root). The pos array maps vertices to a continuous segment workspace. Pass your standard segTree template into query_path and update_path to run path computations.

3.9 Hopcroft-Karp Algorithm

Finds a maximum cardinality matching in a bipartite graph in $O(E\sqrt{V})$ using BFS and DFS step alternations.

```

const int INF = 1e9;

struct HopcroftKarp {
    int n, m;
    vector<vector<int>>> adj;
    vector<int> pair_u, pair_v, dist;

    HopcroftKarp(int n, int m) : n(n), m(m), adj(n + 1), pair_u(n + 1, 0), pair_v(m + 1, 0), dist(n + 1, 0) {}

    void add_edge(int u, int v) {
        adj[u].push_back(v);
    }
};

```



```

}

bool bfs() {
    queue<int> q;
    for (int u = 1; u <= n; u++) {
        if (pair_u[u] == 0) {
            dist[u] = 0;
            q.push(u);
        } else {
            dist[u] = INF;
        }
    }
    dist[0] = INF;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (dist[u] < dist[0]) {
            for (int v : adj[u]) {
                if (dist[pair_v[v]] == INF) {
                    dist[pair_v[v]] = dist[u] + 1;
                    q.push(pair_v[v]);
                }
            }
        }
    }
    return dist[0] != INF;
}

bool dfs(int u) {
    if (u != 0) {
        for (int v : adj[u]) {
            if (dist[pair_v[v]] == dist[u] + 1) {
                if (dfs(pair_v[v])) {
                    pair_v[v] = u;
                    pair_u[u] = v;
                    return true;
                }
            }
        }
        dist[u] = INF;
        return false;
    }
    return true;
}

int max_matching() {
    int matching = 0;
    while (bfs()) {
        for (int u = 1; u <= n; u++) {
            if (pair_u[u] == 0 && dfs(u)) {
                matching++;
            }
        }
    }
    return matching;
}
};

```

3.10 Kruskal Reconstruction Tree (KRT)

A tree structure built on top of DSU by processing edges in increasing order of weight. It allows path max/min edge queries between any two vertices to be reduced to an LCA query in $O(1)$ after preprocessing.

```

struct KRT {
    int n, node_cnt;
    vector<int> parent, sz, node_weight;
    vector<vector<int>> adj;

    KRT(int n) : n(n), node_cnt(n), parent(2 * n + 1), sz(2 * n + 1, 1), node_weight(2 * n + 1, 0), adj(2 * n + 1) {

```

```

        iota(parent.begin(), parent.end(), 0);
    }

    int find_set(int v) {
        if (v == parent[v]) return v;
        return parent[v] = find_set(parent[v]);
    }

    void build(vector<tuple<int, int, int>>& edges) {
        // Sort edges by weight for MST order
        sort(edges.begin(), edges.end());

        for (auto [w, u, v] : edges) {
            int root_u = find_set(u);
            int root_v = find_set(v);

            if (root_u != root_v) {
                node_cnt++;
                node_weight[node_cnt] = w;

                parent[root_u] = node_cnt;
                parent[root_v] = node_cnt;

                adj[node_cnt].push_back(root_u);
                adj[node_cnt].push_back(root_v);
            }
        }
    };
};

```

Pass graph edges as {weight, u, v} array to build(edges). The resulting reconstruction tree contains at most $2N - 1$ nodes where original vertices are leaves ($1 \leq i \leq N$). The maximum edge weight on the path between U and V in the original graph is exactly equal to `node_weight[LCA(u, v)]`.

3.11 LCA with Binary Lifting

Lowest Common Ancestor (LCA) queries on a tree in $O(\log N)$ with $O(N \log N)$ preprocessing using binary lifting.

```

struct LCA {
    int n, l;
    vector<int> tin, tout, depth;
    vector<vector<int>> up;
    int timer;

    LCA(int n, int root, const vector<vector<int>>& adj) : n(n), timer(0) {
        l = __lg(n) + 1;
        tin.resize(n + 1); tout.resize(n + 1); depth.resize(n + 1);
        up.assign(n + 1, vector<int>(l + 1));
        dfs(root, root, 0, adj);
    }

    void dfs(int u, int p, int d, const vector<vector<int>>& adj) {
        tin[u] = ++timer;
        depth[u] = d;
        up[u][0] = p;
        for (int i = 1; i <= l; ++i) {
            up[u][i] = up[up[u][i - 1]][i - 1];
        }
        for (int v : adj[u]) {
            if (v != p) dfs(v, u, d + 1, adj);
        }
        tout[u] = ++timer;
    }

    bool is_ancestor(int u, int v) {
        return tin[u] <= tin[v] && tout[u] >= tout[v];
    }
};

```

```

int query(int u, int v) {
    if (is_ancestor(u, v)) return u;
    if (is_ancestor(v, u)) return v;
    for (int i = l; i >= 0; --i) {
        if (!is_ancestor(up[u][i], v)) u = up[u][i];
    }
    return up[u][0];
}

int get_dist(int u, int v) {
    return depth[u] + depth[v] - 2 * depth[query(u, v)];
}
};

```

3.12 LCA in $O(1)$ with RMQ

Lowest Common Ancestor (LCA) queries on a tree in $O(1)$ with $O(N \log N)$ preprocessing using Euler Tour and Sparse Table.

```

struct LCA_01 {
    vector<int> euler, first, depth;
    vector<vector<int>> st;
    int n, K;

    LCA_01(int n, int root, const vector<vector<int>>& adj) : n(n) {
        first.resize(n + 1);
        depth.resize(n + 1);
        euler.reserve(2 * n);
        dfs(root, 0, 0, adj);

        int m = euler.size();
        K = __lg(m);
        st.assign(K + 1, vector<int>(m));
        for (int i = 0; i < m; i++) st[0][i] = euler[i];

        for (int j = 1; j <= K; j++) {
            for (int i = 0; i + (1 << j) <= m; i++) {
                int u = st[j - 1][i], v = st[j - 1][i + (1 << (j - 1))];
                st[j][i] = (depth[u] < depth[v]) ? u : v;
            }
        }
    }

    void dfs(int u, int p, int d, const vector<vector<int>>& adj) {
        first[u] = euler.size();
        euler.push_back(u);
        depth[u] = d;
        for (int v : adj[u]) {
            if (v != p) {
                dfs(v, u, d + 1, adj);
                euler.push_back(u);
            }
        }
    }

    int query(int u, int v) {
        int l = first[u], r = first[v];
        if (l > r) swap(l, r);
        int j = __lg(r - l + 1);
        int node1 = st[j][l], node2 = st[j][r - (1 << j) + 1];
        return (depth[node1] < depth[node2]) ? node1 : node2;
    }

    int get_dist(int u, int v) {
        return depth[u] + depth[v] - 2 * depth[query(u, v)];
    }
};

```

3.13 Boruvka's Algorithm

Finds the Minimum Spanning Tree (MST) of a graph in $O(E \log V)$. It is highly efficient for parallelization and maps components iteratively using cheapest-edge contraction.

```
struct Boruvka {
    struct Edge { int u, v; long long weight; int id; };
    int n;
    vector<Edge> edges;
    vector<int> parent, sz;

    Boruvka(int n) : n(n), parent(n + 1), sz(n + 1, 1) {
        iota(parent.begin(), parent.end(), 0);
    }

    void add_edge(int u, int v, long long w, int id = 0) {
        edges.push_back({u, v, w, id});
    }

    int find_set(int v) {
        if (v == parent[v]) return v;
        return parent[v] = find_set(parent[v]);
    }

    bool union_sets(int a, int b) {
        a = find_set(a); b = find_set(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        parent[b] = a; sz[a] += sz[b];
        return true;
    }

    long long solve(vector<int>& mst_edges) {
        long long total_weight = 0;
        int num_components = n;
        vector<int> cheapest(n + 1);

        while (num_components > 1) {
            fill(cheapest.begin(), cheapest.end(), -1);
            for (int i = 0; i < edges.size(); i++) {
                int set_u = find_set(edges[i].u);
                int set_v = find_set(edges[i].v);
                if (set_u == set_v) continue;

                if (cheapest[set_u] == -1 || edges[i].weight <
                    edges[cheapest[set_u]].weight) {
                    cheapest[set_u] = i;
                }
                if (cheapest[set_v] == -1 || edges[i].weight <
                    edges[cheapest[set_v]].weight) {
                    cheapest[set_v] = i;
                }
            }

            bool flag = false;
            for (int i = 1; i <= n; i++) {
                if (cheapest[i] != -1) {
                    int idx = cheapest[i];
                    if (union_sets(edges[idx].u, edges[idx].v)) {
                        total_weight += edges[idx].weight;
                        mst_edges.push_back(edges[idx].id);
                        num_components--;
                        flag = true;
                    }
                }
            }
            if (!flag) break; // Graph is disconnected
        }
        return (num_components == 1) ? total_weight : -1;
    }
};
```

Initialize via Boruvka solver(n), push edges, and call solver.solve(mst_edges). It returns the minimum weight sum and populates the vector with selected edge IDs.

3.14 Kruskal's Algorithm

Finds the Minimum Spanning Tree (MST) of a graph in $O(E \log E)$ by sorting all edges and greedily merging components using a Disjoint Set Union (DSU) structure.

```
struct Edge {
    int u, v, weight, id;
    bool operator*(const Edge& other) const {
        return weight < other.weight;
    }
};

struct KruskalDSU {
    vector<int> parent, sz;
    KruskalDSU(int n) {
        parent.resize(n + 1); sz.assign(n + 1, 1);
        iota(parent.begin(), parent.end(), 0);
    }
    int find_set(int v) {
        if (v == parent[v]) return v;
        return parent[v] = find_set(parent[v]);
    }
    bool union_sets(int a, int b) {
        a = find_set(a); b = find_set(b);
        if (a != b) {
            if (sz[a] < sz[b]) swap(a, b);
            parent[b] = a; sz[a] += sz[b];
            return true;
        }
        return false;
    }
};

long long kruskal(int n, vector<Edge>& edges, vector<int>&
    mst_edges) {
    sort(edges.begin(), edges.end());
    KruskalDSU dsu(n);
    long long total_weight = 0;
    int edges_counted = 0;

    for (const auto& edge : edges) {
        if (dsu.union_sets(edge.u, edge.v)) {
            total_weight += edge.weight;
            mst_edges.push_back(edge.id);
            edges_counted++;
            if (edges_counted == n - 1) break;
        }
    }
    return (edges_counted == n - 1) ? total_weight : -1;
}
```

To find the MST, store all elements in a flat vector<Edge> array. Call kruskal(n, edges, mst_edges) to compute the minimum total weight and populate the vector with selected edge IDs.

3.15 Prim's Algorithm

Finds the Minimum Spanning Tree (MST) of a graph from a starting vertex in $O(E \log V)$ using a priority queue. It is optimal for dense graphs and grows a single tree component.

```
const long long INF = 1e18;
struct Edge { int to, weight, id; };
```

```
long long prim(int source, int n, const vector<vector<Edge>>& adj,
    vector<int>& mst_edges) {
    long long total_weight = 0;
    vector<bool> visited(n + 1, false);
    vector<long long> min_edge(n + 1, INF);
    vector<int> parent_edge(n + 1, -1);

    // priority_queue stores {weight, vertex}
    priority_queue<pair<long long, int>, vector<pair<long long, int>
        >>, greater<pair<long long, int>>> pq;

    min_edge[source] = 0;
    pq.push({0, source});
    int visited_count = 0;

    while (!pq.empty()) {
        auto [w, u] = pq.top(); pq.pop();
        if (visited[u]) continue;

        visited[u] = true;
        total_weight += w;
        visited_count++;

        if (parent_edge[u] != -1) {
            mst_edges.push_back(parent_edge[u]);
        }

        for (const auto& edge : adj[u]) {
            if (!visited[edge.to] && edge.weight < min_edge[edge.to]) {
                min_edge[edge.to] = edge.weight;
                parent_edge[edge.to] = edge.id;
                pq.push({min_edge[edge.to], edge.to});
            }
        }
    }
    return (visited_count == n) ? total_weight : -1;
}
```

To find the MST, build your adjacency list passing an incremental edge ID into Edge.id. Call prim(1, n, adj, mst_edges) to compute the minimum total weight and populate the vector with selected edge IDs.

3.16 Kosaraju's Algorithm

Finds Strongly Connected Components (SCC) in a directed graph in $O(V + E)$ using two passes of Depth First Search (DFS).

```
struct Kosaraju {
    int n, scc_count;
    vector<vector<int>> adj, adj_rev;
    vector<bool> visited;
    vector<int> order, scc_id;

    Kosaraju(int n) : n(n), scc_count(0), adj(n + 1), adj_rev(n + 1),
        visited(n + 1, false), scc_id(n + 1, -1) {}

    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj_rev[v].push_back(u);
    }

    void dfs1(int u) {
        visited[u] = true;
        for (int v : adj[u]) {
            if (!visited[v]) dfs1(v);
        }
        order.push_back(u);
    }
};
```

```

void dfs2(int u, int c1) {
    visited[u] = true;
    scc_id[u] = c1;
    for (int v : adj_rev[u]) {
        if (!visited[v]) dfs2(v, c1);
    }
}

void run() {
    for (int i = 1; i <= n; i++) {
        if (!visited[i]) dfs1(i);
    }
    fill(visited.begin(), visited.end(), false);
    reverse(order.begin(), order.end());
    for (int u : order) {
        if (!visited[u]) {
            dfs2(u, ++scc_count);
        }
    }
}
};

```

Call `kosaraju.run()` to compute components. The total number of SCCs is stored in `scc_count`, and the 1-based component ID for each vertex is accessible via `scc_id[i]`.

3.17 Tarjan Algorithm

Finds Strongly Connected Components (SCC) in a directed graph, or bridges and articulation points in an undirected graph in $O(V + E)$ using a single DFS traversal.

```

struct Tarjan {
    int n, timer, scc_count;
    vector<vector<int>> adj;
    vector<int> tin, low, scc_id;
    vector<bool> in_stack;
    stack<int> st;

    Tarjan(int n, const vector<vector<int>>& adj) :
        n(n), adj(adj), timer(0), scc_count(0),
        tin(n + 1, -1), low(n + 1, -1), scc_id(n + 1, -1), in_stack
        (n + 1, false) {}

    void dfs(int u) {
        tin[u] = low[u] = ++timer;
        st.push(u);
        in_stack[u] = true;

        for (int v : adj[u]) {
            if (tin[v] == -1) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (in_stack[v]) {
                low[u] = min(low[u], tin[v]);
            }
        }

        if (low[u] == tin[u]) {
            scc_count++;
            while (true) {
                int v = st.top(); st.pop();
                in_stack[v] = false;
                scc_id[v] = scc_count;
                if (u == v) break;
            }
        }
    }

    void run() {

```

```

        for (int i = 1; i <= n; i++) {
            if (tin[i] == -1) dfs(i);
        }
    }
};

```

3.18 Virtual Tree (Auxiliary Tree)

Constructs a compressed tree containing only a specific subset of "k" marked nodes and their essential Lowest Common Ancestors (LCA) in $O(K \log K)$.

```

struct VirtualTree {
    LCA &lca;
    vector<vector<int>> v_adj;

    VirtualTree(LCA &lca) : lca(lca), v_adj(lca.n + 1) {}

    void build(vector<int> &k_nodes) {
        sort(k_nodes.begin(), k_nodes.end(), [&](int a, int b) {
            return lca.tin[a] < lca.tin[b];
        });

        int m = k_nodes.size();
        for (int i = 0; i < m - 1; i++) {
            int anc = lca.query(k_nodes[i], k_nodes[i + 1]);
            k_nodes.push_back(anc);
        }
        sort(k_nodes.begin(), k_nodes.end(), [&](int a, int b) {
            return lca.tin[a] < lca.tin[b];
        });
        k_nodes.erase(unique(k_nodes.begin(), k_nodes.end()),
            k_nodes.end());

        for (int u : k_nodes) v_adj[u].clear();

        vector<int> st;
        for (int u : k_nodes) {
            while (!st.empty() && !lca.is_ancestor(st.back(), u)) {
                st.pop_back();
            }
            if (!st.empty()) {
                v_adj[st.back()].push_back(u);
            }
            st.push_back(u);
        }
    }
};

```

Initialize via `VirtualTree vt(lca_instance)` and call `vt.build(k_nodes)`. The resulting compressed hierarchy is populated directly inside `vt.v_adj`.

4 Math

4.1 Chinese Remainder Theorem (CRT)

Solves a system of simultaneous modular congruences $x \equiv r_i \pmod{m_i}$ for pairwise coprime moduli in $O(N \log(\max M_i))$ time complexity using the Extended Euclidean Algorithm.

```

long long extgcd(long long a, long long b, long long &x, long long
&y) {
    if (b == 0) { x = 1; y = 0; return a; }
    long long x1, y1;
    long long d = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
}

```

```

    return d;
}

// Solves a system of 2 congruences: x = r1 (mod m1) and x = r2 (
mod m2).
// Works even if m1 and m2 are NOT coprime. Returns {-1, -1} if no
solution exists.
pair<long long, long long> crt_two(long long r1, long long m1, long
long r2, long long m2) {
    long long x, y;
    long long g = extgcd(m1, m2, x, y);
    if ((r2 - r1) % g != 0) return {-1, -1};

    long long lcm = m1 / g * m2;
    long long mod = m2 / g;
    long long x0 = ((x % mod) + mod) % mod;
    long long factor = (r2 - r1) / g;
    long long ans = (r1 + (__int128)factor * x0 % lcm * m1) % lcm;
    if (ans < 0) ans += lcm;
    return {ans, lcm};
}

// Solves a system of N congruences. Returns {-1, -1} if no
solution exists.
// Input format: vectors of pairs {remainder, modulus}
pair<long long, long long> crt(const vector<pair<long long, long
long>>& equations) {
    if (equations.empty()) return {0, 1};
    long long r_ans = equations[0].first;
    long long m_ans = equations[0].second;

    for (size_t i = 1; i < equations.size(); i++) {
        auto [r, m] = crt_two(r_ans, m_ans, equations[i].first,
equations[i].second);
        if (m == -1) return {-1, -1};
        r_ans = r;
        m_ans = m;
    }
    return {r_ans, m_ans};
}

```

To find a number x that satisfies $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, and $x \equiv 2 \pmod{7}$:

```

vector<pair<long long, long long>> eq = {{2, 3}, {3, 5}, {2, 7}};
pair<long long, long long> ans = crt(eq);
if (ans.second != -1) {
    long long x = ans.first; // Unique solution modulo LCM (23)
    long long lcm = ans.second; // LCM of all moduli (105)
}

```

4.2 Euler's Totient Function (Phi)

Computes Euler's totient function $\phi(n)$, which counts the number of integers up to n that are relatively prime to n . Single queries run in $O(\sqrt{N})$, and precomputation for a range $[1, N]$ runs in $O(N \log \log N)$ using a sieve.

```

// 1. Single value query in O(sqrt(N))
long long phi_single(long long n) {
    long long res = n;
    for (long long i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0) n /= i;
            res -= res / i;
        }
    }
    if (n > 1) res -= res / n;
    return res;
}

```

```
// 2. Precompute phi for all numbers from 1 to N using Sieve
vector<int> phi_sieve(int n) {
    vector<int> phi(n + 1);
    for (int i = 0; i <= n; i++) phi[i] = i;
    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) { // i is prime
            for (int j = i; j <= n; j += i) {
                phi[j] -= phi[j] / i;
            }
        }
    }
    return phi;
}
```

Use `phi_single(n)` for isolated large values. For multiple queries up to N , run `vector<int> phi = phi_sieve(N)` during initialization to fetch results in $O(1)$.

4.3 Extended Euclidean

Computes the greatest common divisor of a and b , and finds integers x and y such that $ax + by = \gcd(a, b)$ in $O(\log(\min(a, b)))$. `extgcd` returns the $\gcd(a, b)$ and updates x and y via references.

```
long long extgcd(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    long long x1, y1;
    long long d = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}
```

To find the modular inverse of a modulo m using this algorithm, ensure $\gcd(a, m) = 1$. The inverse exists if $d = 1$, and it can be computed as $(x \% m + m) \% m$ after calling `extgcd(a, m, x, y)`.

4.4 FFT & NTT (Fast Fourier Transform & Number Theoretic Transform)

Polynomial multiplication in $O(N \log N)$. Use `FFT` for high-precision floating-point coefficients and `NTT` for exact integer multiplication under a large prime modulus (typically 998244353).

```
const double PI = acos(-1);
struct Complex {
    double r, i;
    Complex(double r = 0, double i = 0) : r(r), i(i) {}
    Complex operator+(const Complex& o) const { return {r + o.r, i + o.i}; }
    Complex operator-(const Complex& o) const { return {r - o.r, i - o.i}; }
    Complex operator*(const Complex& o) const { return {r * o.r - i * o.i, r * o.i + i * o.r}; }
};

void fft(vector<Complex>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
    }
    if (invert) {
        long long n_inv = power(n, MOD - 2);
        for (auto& x : a) x = (x * n_inv) % MOD;
    }
}
```

```
j ^= bit;
if (i < j) swap(a[i], a[j]);
}
for (int len = 2; len <= n; len <<= 1) {
    double ang = 2 * PI / len * (invert ? -1 : 1);
    Complex wlen(cos(ang), sin(ang));
    for (int i = 0; i < n; i += len) {
        Complex w(1);
        for (int j = 0; j < len / 2; j++) {
            Complex u = a[i + j], v = a[i + j + len / 2] * w;
            a[i + j] = u + v;
            a[i + j + len / 2] = u - v;
            w = w * wlen;
        }
    }
}
if (invert) {
    for (auto& x : a) x.r /= n;
}
}

vector<long long> multiply_fft(const vector<int>& a, const vector<int>& b) {
    vector<Complex> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1; while (n < a.size() + b.size()) n <<= 1;
    fa.resize(n); fb.resize(n);
    fft(fa, false); fft(fb, false);
    for (int i = 0; i < n; i++) fa[i] = fa[i] * fb[i];
    fft(fa, true);
    vector<long long> res(n);
    for (int i = 0; i < n; i++) res[i] = round(fa[i].r);
    return res;
}
```

```
const int MOD = 998244353;
const int G = 3; // Primitive root for 998244353
```

```
long long power(long long base, long long p) {
    long long res = 1; base %= MOD;
    while (p > 0) {
        if (p & 1) res = (res * base) % MOD;
        base = (base * base) % MOD;
        p >>= 1;
    }
    return res;
}

void ntt(vector<long long>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        long long wlen = power(G, (MOD - 1) / len);
        if (invert) wlen = power(wlen, MOD - 2);
        for (int i = 0; i < n; i += len) {
            long long w = 1;
            for (int j = 0; j < len / 2; j++) {
                long long u = a[i + j], v = (a[i + j + len / 2] * w) % MOD;
                a[i + j] = (u + v < MOD ? u + v : u + v - MOD);
                a[i + j + len / 2] = (u - v >= 0 ? u - v : u - v + MOD);
                w = (w * wlen) % MOD;
            }
        }
    }
    if (invert) {
        long long n_inv = power(n, MOD - 2);
        for (auto& x : a) x = (x * n_inv) % MOD;
    }
}
```

```
}
}

vector<long long> multiply_ntt(const vector<long long>& a, const vector<long long>& b) {
    vector<long long> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    int n = 1; while (n < a.size() + b.size()) n <<= 1;
    fa.resize(n, 0); fb.resize(n, 0);
    ntt(fa, false); ntt(fb, false);
    for (int i = 0; i < n; i++) fa[i] = (fa[i] * fb[i]) % MOD;
    ntt(fa, true);
    return fa;
}
```

To multiply two polynomials $A(x)$ and $B(x)$, pass their coefficients to `multiply_fft` or `multiply_ntt`. For example:

```
// Multiply (1 + 2x) * (2 + 3x) -> 2 + 7x + 6x^2
vector<int> a = {1, 2};
vector<int> b = {2, 3};
vector<long long> c = multiply_fft(a, b);
// c contains: {2, 7, 6, 0, ...}
```

4.5 Matrix Exponentiation Template

A complete matrix template for solving recurrence relations and dynamic programming optimizations in $O(K^3 \log N)$, featuring modular addition, subtraction, multiplication, and fast power operations.

```
const int MOD = 1e9 + 7;

struct Matrix {
    int r, c;
    vector<vector<long long>> mat;

    Matrix(int r, int c, bool identity = false) : r(r), c(c), mat(r, vector<long long>(c, 0)) {
        if (identity) {
            for (int i = 0; i < min(r, c); i++) mat[i][i] = 1;
        }
    }

    Matrix operator+(const Matrix& other) const {
        Matrix res(r, c);
        for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                res.mat[i][j] = (mat[i][j] + other.mat[i][j]) % MOD;
            }
        }
        return res;
    }

    Matrix operator-(const Matrix& other) const {
        Matrix res(r, c);
        for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                res.mat[i][j] = (mat[i][j] - other.mat[i][j] + MOD) % MOD;
            }
        }
        return res;
    }

    Matrix operator*(const Matrix& other) const {
        Matrix res(r, other.c);
        for (int i = 0; i < r; i++) {
            for (int k = 0; k < c; k++) {
                for (int j = 0; j < other.c; j++) {
                    res.mat[i][j] = (res.mat[i][j] + mat[i][k] * other.mat[k][j]) % MOD;
                }
            }
        }
        return res;
    }
}
```

```

        for (int j = 0; j < other.c; j++) {
            res.mat[i][j] = (res.mat[i][j] + mat[i][k] *
                other.mat[k][j]) % MOD;
        }
    }
    return res;
}

Matrix power(long long p) const {
    Matrix res(r, r, true);
    Matrix base = *this;
    while (p > 0) {
        if (p & 1) res = res * base;
        base = base * base;
        p >>= 1;
    }
    return res;
}
};

```

To initialize a matrix with custom values or solve Fibonacci $F_n = F_{n-1} + F_{n-2}$, use the following initialization pattern:

```

// 1. Initialize a 2x2 transition matrix T
Matrix T(2, 2);
T.mat = {
    {1, 1},
    {1, 0}
};

// 2. Initialize a 2x1 base state column vector F
Matrix F(2, 1);
F.mat = {
    {1}, // F[1] = 1
    {0}  // F[0] = 0
};

// 3. Compute T^(n-1) * F to get F[n]
Matrix Tn = T.power(n - 1);
Matrix Ans = Tn * F;
long long fn = Ans.mat[0][0];

```

4.6 Miller-Rabin Primality Test & Pollard's Rho

Deterministic Miller-Rabin primality test for 64-bit integers in $O(K \log^3 N)$ alongside Pollard's Rho algorithm for prime factorization in $O(N^{1/4})$.

```

typedef unsigned long long ull;

ull mul_mod(ull a, ull b, ull m) {
    long long res = a * b - m * ((long double)a * b / m);
    return (res < 0 ? res + m : res) % m;
}

ull power(ull base, ull p, ull mod) {
    ull res = 1; base %= mod;
    while (p > 0) {
        if (p & 1) res = mul_mod(res, base, mod);
        base = mul_mod(base, base, mod);
        p >>= 1;
    }
    return res;
}

bool miller_rabin(ull n) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0) return false;

```

```

    ull d = n - 1; int s = 0;
    while (d % 2 == 0) { d /= 2; s++; }
    static const ull bases[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37};
    for (ull a : bases) {
        if (n <= a) break;
        ull x = power(a, d, n);
        if (x == 1 || x == n - 1) continue;
        bool composite = true;
        for (int r = 1; r < s; r++) {
            x = mul_mod(x, x, n);
            if (x == n - 1) { composite = false; break; }
        }
        if (composite) return false;
    }
    return true;
}

ull pollard_rho(ull n) {
    if (n % 2 == 0) return 2;
    if (miller_rabin(n)) return n;
    ull x = 2, y = 2, d = 1, c = 1;
    auto f = [&](ull x, ull n, ull c) { return (mul_mod(x, x, n) +
        c) % n; };
    while (d == 1) {
        x = f(x, n, c); y = f(f(y, n, c), n, c);
        d = std::gcd(x > y ? x - y : y - x, n);
        if (d == n) { x = rand() % (n - 2) + 2; y = x; c = rand() %
            (n - 1) + 1; d = 1; }
    }
    return d;
}

void factorize(ull n, vector<ull>& factors) {
    if (n == 1) return;
    if (miller_rabin(n)) { factors.push_back(n); return; }
    ull p = pollard_rho(n);
    factorize(p, factors);
    factorize(n / p, factors);
}

```

Use miller_rabin(n) to check primality. To find all prime factors, use factorize(n, factors) then sort(factors.begin(), factors.end()).

4.7 Fast Binary GCD (Stein's Algorithm)

An ultra-fast GCD computation running in $O(\log(\max(A, B)))$.

```

long long fast_gcd(long long a, long long b) {
    if (!a || !b) return a ^ b;
    int shift = __builtin_ctzll(a | b);
    a >>= __builtin_ctzll(a);
    while (b) {
        b >>= __builtin_ctzll(b);
        if (a > b) swap(a, b);
        b -= a;
    }
    return a << shift;
}

```

4.8 Tonelli-Shanks Algorithm

Solves the modular congruence $x^2 \equiv n \pmod{p}$ for a prime p in $O(\log^2 p)$ time complexity. It finds the modular square root if it exists.

```

long long power(long long base, long long p, long long mod) {
    long long res = 1; base %= mod;
    while (p > 0) {
        if (p & 1) res = (__int128)res * base % mod;

```

```

        base = (__int128)base * base % mod;
        p >>= 1;
    }
    return res;
}

long long tonelli_shanks(long long n, long long p) {
    if (p == 2) return n & 1;
    if (power(n, (p - 1) / 2, p) != 1) return -1; // No solution (Legendre symbol)
    if (p % 4 == 3) return power(n, (p + 1) / 4, p);

    long long q = p - 1, s = 0;
    while (q % 2 == 0) { q /= 2; s++; }

    long long z = 2;
    while (power(z, (p - 1) / 2, p) == 1) z++;

    long long m = s, c = power(z, q, p), t = power(n, q, p), r =
        power(n, (q + 1) / 2, p);

    while (t != 1) {
        long long t2k = t, k = 0;
        for (k = 1; k < m; k++) {
            t2k = (__int128)t2k * t2k % p;
            if (t2k == 1) break;
        }
        if (k == m) return -1;
        long long b = power(c, 1LL << (m - k - 1), p);
        r = (__int128)r * b % p;
        c = (__int128)b * b % p;
        t = (__int128)t * c % p;
        m = k;
    }
    return r;
}

```

Call tonelli_shanks(n, p) to find a value x such that $x^2 \equiv n \pmod{p}$.

- Returns a valid solution $x \in [0, p - 1]$ if it exists.
- Returns -1 if n is a quadratic non-residue modulo p (no solution exists).
- Note that if x is a solution, $p - x$ is also a valid solution.

5 Strings

5.1 Aho-Corasick Automaton

Multi-pattern string matching automaton built on top of a Trie. It constructs suffix links and dictionary links using BFS in $O(\sum |pattern|)$ time, allowing simultaneous matching of multiple strings in $O(|text|)$.

```

struct AhoCorasick {
    static const int ALPHABET_SIZE = 26;
    struct Node {
        int next[ALPHABET_SIZE];
        int link = -1; // Suffix link
        int exit_link = -1; // Dict link to nearest pattern end
    };
    vector<int> pattern_ids; // IDs of patterns ending at this node
    Node() { memset(next, -1, sizeof(next)); }
};

```

```

vector<Node> trie;
AhoCorasick() { trie.push_back(Node()); }

void insert(const string& s, int id) {
    int u = 0;
    for (char c : s) {
        int idx = c - 'a';
        if (trie[u].next[idx] == -1) {
            trie[u].next[idx] = trie.size();
            trie.push_back(Node());
        }
        u = trie[u].next[idx];
    }
    trie[u].pattern_ids.push_back(id);
}

void build() {
    queue<int> q;
    for (int i = 0; i < ALPHABET_SIZE; i++) {
        if (trie[0].next[i] != -1) {
            trie[trie[0].next[i]].link = 0;
            q.push(trie[0].next[i]);
        } else {
            trie[0].next[i] = 0; // Optimization: transform
            // Trie into a DFA transition table
        }
    }

    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int i = 0; i < ALPHABET_SIZE; i++) {
            int v = trie[u].next[i];
            if (v != -1) {
                trie[v].link = trie[trie[u].link].next[i];

                // Construct dictionary exit link
                int slink = trie[v].link;
                if (!trie[slink].pattern_ids.empty()) trie[v].
                exit_link = slink;
                else trie[v].exit_link = trie[slink].exit_link;

                q.push(v);
            } else {
                trie[u].next[i] = trie[trie[u].link].next[i];
            }
        }
    }
}
};

```

Initialize via `AhoCorasick ac`, call `ac.insert(s, id)` for all patterns, and then run `ac.build()` once. When parsing a text, traverse using the modified transition matrix `u = ac.trie[u].next[c - 'a']`. Use the `exit_link` chain at each step to collect all matched dictionary patterns efficiently.

5.2 KMP (Knuth-Morris-Pratt) Algorithm

Pattern matching in $O(N + M)$ time complexity using a prefix function (`pi` array).

```

vector<int> prefix_function(string s) {
    int n = s.length();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
}

```

```

    return pi;
}

vector<int> kmp_search(string text, string pattern) {
    string s = pattern + "#" + text;
    int n = s.length(), m = pattern.length();
    vector<int> pi = prefix_function(s);
    vector<int> matches;
    for (int i = m + 1; i < n; i++) {
        if (pi[i] == m) {
            matches.push_back(i - 2 * m); // 0-indexed position in
            text
        }
    }
    return matches;
}

```

To find all occurrences of a pattern inside a text, call `kmp_search(text, pattern)`. It returns a vector containing the starting 0-indexed positions of all valid matches.

5.3 Manacher's Algorithm

Computes the longest palindromic substring centered at each position of a string in $O(N)$ time by utilizing previously calculated symmetric palindrome radius bounds.

```

vector<int> manacher(string s) {
    string t = "#";
    for (char c : s) {
        t += c;
        t += "#";
    }
    int n = t.length();
    vector<int> p(n, 0);
    int c = 0, r = 0;

    for (int i = 0; i < n; i++) {
        int mirror = 2 * c - i;
        if (i < r) {
            p[i] = min(r - i, p[mirror]);
        }
        while (i + p[i] + 1 < n && i - p[i] - 1 >= 0 && t[i + p[i]
        + 1] == t[i - p[i] - 1]) {
            p[i]++;
        }
        if (i + p[i] > r) {
            c = i;
            r = i + p[i];
        }
    }
    return p;
}

```

Pass a string `s` to `manacher(s)`. The function internally pads the string with `'#'` characters to uniformly handle both odd and even length palindromes. The actual length of the maximum palindrome centered at index `i` in the transformed array is exactly equal to `p[i]`.

5.4 Palindrome Tree (Eertree)

A specialized tree structure that processes characters online to maintain all unique palindromic substrings of a string in $O(N)$ time and space complexity.

```

struct PalindromeTree {

```

```

    static const int ALPHABET_SIZE = 26;
    struct Node {
        int next[ALPHABET_SIZE];
        int len;
        int link;
        int cnt; // Frequency of the palindrome substring
    };

    string s;
    vector<Node> tree;
    int sz, last;

    PalindromeTree() {
        s = "";
        // Node 1: Imaginary root of odd length (-1)
        // Node 2: Root of even length (0)
        tree.push_back({}); // Dummy at index 0
        tree.push_back({}); tree[1].len = -1; tree[1].link = 1;
        tree.push_back({}); tree[2].len = 0; tree[2].link = 1;
        memset(tree[1].next, 0, sizeof(tree[1].next));
        memset(tree[2].next, 0, sizeof(tree[2].next));
        sz = 2; last = 2;
    }

    int get_link(int u) {
        int curr_pos = s.length() - 1;
        while (true) {
            int prev_pos = curr_pos - 1 - tree[u].len;
            if (prev_pos >= 0 && s[prev_pos] == s[curr_pos]) return
            u;
            u = tree[u].link;
        }
    }

    void insert(char ch) {
        s += ch;
        int c = ch - 'a';
        int curr = get_link(last);

        if (tree[curr].next[c]) {
            last = tree[curr].next[c];
            tree[last].cnt++;
            return;
        }

        sz++;
        last = sz;
        tree.push_back({});
        memset(tree[sz].next, 0, sizeof(tree[sz].next));

        tree[sz].len = tree[curr].len + 2;
        tree[curr].next[c] = sz;

        if (tree[sz].len == 1) {
            tree[sz].link = 2;
            tree[sz].cnt = 1;
            return;
        }

        int link_node = get_link(tree[curr].link);
        tree[sz].link = tree[link_node].next[c];
        tree[sz].cnt = 1;
    }

    void pull_frequencies() {
        // Propagate counts from longer palindromes to their suffix
        links
        for (int i = sz; i > 2; i--) {
            tree[tree[i].link].cnt += tree[i].cnt;
        }
    }
};

```

Initialize via `PalindromeTree pt` and append characters one by one

using `pt.insert(c)`. Call `pt.pull_frequencies()` at the end to get the exact total occurrence count of every unique palindromic substring inside `tree[i].cnt`.

5.5 String Double Hashing

Anti-hash-kill String Hashing implementation using dynamic base generation and double rolling hash matching under random secure primes.

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

struct StringHash {
    long long MOD1 = 1e9 + 7, MOD2 = 1e9 + 9;
    long long BASE1, BASE2;
    vector<long long> h1, h2, p1, p2;

    StringHash(const string& s) {
        BASE1 = (rng() % (MOD1 - 300) + 300) | 1;
        BASE2 = (rng() % (MOD2 - 300) + 300) | 1;
        int n = s.size();
        h1.resize(n + 1, 0); h2.resize(n + 1, 0);
        p1.resize(n + 1, 1); p2.resize(n + 1, 1);
        for (int i = 0; i < n; i++) {
            h1[i + 1] = (h1[i] * BASE1 + s[i]) % MOD1;
            h2[i + 1] = (h2[i] * BASE2 + s[i]) % MOD2;
            p1[i + 1] = (p1[i] * BASE1) % MOD1;
            p2[i + 1] = (p2[i] * BASE2) % MOD2;
        }
    }

    pair<long long, long long> get(int l, int r) {
        l++; r++;
        long long res1 = (h1[r] - h1[l - 1] * p1[r - l + 1]) % MOD1;
        long long res2 = (h2[r] - h2[l - 1] * p2[r - l + 1]) % MOD2;
        if (res1 < 0) res1 += MOD1;
        if (res2 < 0) res2 += MOD2;
        return {res1, res2};
    }
};
```

To verify matching substrings over range $[l, r]$, initialize via `StringHash hasher(s)` and query using 0-indexed values `hasher.get(l, r)`.

5.6 String Single Hashing

Anti-hash-kill String Hashing implementation using dynamic base generation under a single secure random prime.

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

struct StringHash {
    long long MOD = 1e9 + 7;
    long long BASE;
    vector<long long> h, p;

    StringHash(const string& s) {
        BASE = (rng() % (MOD - 300) + 300) | 1;
        int n = s.size();
        h.resize(n + 1, 0);
        p.resize(n + 1, 1);
        for (int i = 0; i < n; i++) {
            h[i + 1] = (h[i] * BASE + s[i]) % MOD;
            p[i + 1] = (p[i] * BASE) % MOD;
        }
    }
};
```

```
long long get(int l, int r) {
    l++; r++;
    long long res = (h[r] - h[l - 1] * p[r - l + 1]) % MOD;
    if (res < 0) res += MOD;
    return res;
}

};
```

To verify matching substrings over range $[l, r]$, initialize via `StringHash hasher(s)` and query using 0-indexed values `hasher.get(l, r)`.

5.7 Suffix Array

Constructs the Suffix Array of a string in $O(N \log N)$ time complexity using radix sort simulation on prefix ranks.

```
struct SuffixArray {
    string s;
    int n;
    vector<int> sa, rank, lcp;

    SuffixArray(string s) : s(s + "$"), n(s.length() + 1) {
        sa.resize(n); rank.resize(n);
        build_sa();
        build_lcp();
    }

    void build_sa() {
        vector<int> p(n), c(n), cnt(max(256, n), 0);
        for (int i = 0; i < n; i++) cnt[s[i]]++;
        for (int i = 1; i < 256; i++) cnt[i] += cnt[i - 1];
        for (int i = 0; i < n; i++) p[--cnt[s[i]]] = i;
        c[p[0]] = 0;
        int classes = 1;
        for (int i = 1; i < n; i++) {
            if (s[p[i]] != s[p[i - 1]]) classes++;
            c[p[i]] = classes - 1;
        }
        vector<int> pn(n), cn(n);
        for (int h = 0; (1 << h) < n; ++h) {
            for (int i = 0; i < n; i++) {
                pn[i] = p[i] - (1 << h);
                if (pn[i] < 0) pn[i] += n;
            }
            fill(cnt.begin(), cnt.begin() + classes, 0);
            for (int i = 0; i < n; i++) cnt[c[pn[i]]]++;
            for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
            for (int i = n - 1; i >= 0; i--) p[--cnt[c[pn[i]]]] = pn[i];
            cn[p[0]] = 0;
            classes = 1;
            for (int i = 1; i < n; i++) {
                pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << h)) % n]};
                pair<int, int> prev = {c[p[i - 1]], c[(p[i - 1] + (1 << h)) % n]};
                if (cur != prev) classes++;
                cn[p[i]] = classes - 1;
            }
            c = cn;
        }
        sa = p;
        for (int i = 0; i < n; i++) rank[sa[i]] = i;
    }

    void build_lcp() {
        lcp.assign(n, 0);
        int k = 0;
        for (int i = 0; i < n - 1; i++) {
            int pi = rank[i];
            int j = sa[pi - 1];
            while (s[i + k] == s[j + k]) k++;
        }
    }
};
```

```
while (s[i + k] == s[j + k]) k++;
lcp[pi] = k;
if (k) k--;
}
};
```

Initialize via `SuffixArray sa_struct(s)`. The sentinel character '\$' is appended internally. The sorted suffix indices are available in `sa_struct.sa` starting from index 1, and `sa_struct.lcp[i]` stores the Longest Common Prefix between suffix `sa[i]` and `sa[i-1]`.

5.8 Trie

Prefix tree structure for character string tracking, insertions, and searches in $O(L)$ where L represents string length.

```
struct Trie {
    static const int ALPHABET_SIZE = 26;
    struct TrieNode {
        TrieNode* children[ALPHABET_SIZE];
        bool is_end_of_word;
        TrieNode() {
            is_end_of_word = false;
            for (int i = 0; i < ALPHABET_SIZE; i++) children[i] = nullptr;
        }
    };
    TrieNode* root;
    Trie() { root = new TrieNode(); }

    void insert(const string& key) {
        TrieNode* curr = root;
        for (char c : key) {
            int index = c - 'a';
            if (!curr->children[index]) curr->children[index] = new TrieNode();
            curr = curr->children[index];
        }
        curr->is_end_of_word = true;
    }

    bool search(const string& key) {
        TrieNode* curr = root;
        for (char c : key) {
            int index = c - 'a';
            if (!curr->children[index]) return false;
            curr = curr->children[index];
        }
        return curr->is_end_of_word;
    }
};
```

Initialize via `Trie trie` to store and look up strings containing standard lowercase alphabetical letters.

5.9 Z-Algorithm

Computes the Z-array in $O(N)$ time complexity, where each element $Z[i]$ stores the length of the longest common prefix between the string s and its suffix starting at index i .

```
vector<int> z_function(string s) {
    int n = s.length();
    vector<int> z(n, 0);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
```



```

    if (i <= r) {
        z[i] = min(r - i + 1, z[i - 1]);
    }
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
        z[i]++;
    }
    if (i + z[i] - 1 > r) {
        l = i;
        r = i + z[i] - 1;
    }
}
return z;
}

vector<int> z_search(string text, string pattern) {
    string s = pattern + "#" + text;
    int m = pattern.length();
    vector<int> z = z_function(s);
    vector<int> matches;
    for (int i = m + 1; i < s.length(); i++) {
        if (z[i] == m) {
            matches.push_back(i - m - 1); // 0-indexed position in
            text
        }
    }
    return matches;
}

```

To find all occurrences of a pattern inside a text, call `z_search(text, pattern)`. It utilizes a special separator character like `#` to prevent bounds crossing and returns a vector containing all starting 0-indexed match indices.

6 BitManipulation

6.1 Binary Trie (Bit Trie)

Prefix tree structure for processing 32-bit integers, useful for Max XOR queries or subset elements monitoring in $O(B)$ where B represents the bit-width.

```

struct BitTrie {
    static const int BITS = 30; // 30 bits for values up to 10^9
    struct Node {
        int next[2];
        int cnt;
        Node() { next[0] = next[1] = 0; cnt = 0; }
    };
    vector<Node> trie;

    BitTrie() { trie.push_back(Node()); }

    void insert(int x) {
        int u = 0;
        trie[u].cnt++;
        for (int i = BITS; i >= 0; i--) {
            int bit = (x >> i) & 1;
            if (!trie[u].next[bit]) {
                trie[u].next[bit] = trie.size();
                trie.push_back(Node());
            }
            u = trie[u].next[bit];
            trie[u].cnt++;
        }
    }

    void remove(int x) {
        int u = 0;
        trie[u].cnt--;
        for (int i = BITS; i >= 0; i--) {

```

```

            int bit = (x >> i) & 1;
            u = trie[u].next[bit];
            trie[u].cnt--;
        }
    }

    int max_xor(int x) {
        int u = 0, res = 0;
        for (int i = BITS; i >= 0; i--) {
            int bit = (x >> i) & 1;
            int toggle = bit ^ 1;
            if (trie[u].next[toggle] && trie[trie[u].next[toggle]].
                cnt > 0) {
                res |= (1 << i);
                u = trie[u].next[toggle];
            } else {
                u = trie[u].next[bit];
            }
            return res;
        }
    }
};

```

Initialize via BitTrie trie to insert elements. Use `trie.max_xor(x)` to find the maximum possible value of $x \oplus y$ among all elements y currently stored in the structure.

6.2 Bitwise Hacks

High-performance bitwise manipulation patterns and $O(1)$ hardware accelerated applications.

```

// 1. Core Bit Operations with CP Applications
x | (1LL << i) // Set i-th bit to 1 (State transition
              in Bitmask DP)
x & ~(1LL << i) // Set i-th bit to 0
x ^ (1LL << i) // Toggle i-th bit
(x >> i) & 1LL // Check if i-th bit is 1 (Verify
              subset inclusion)
bool is_pow2 = x && !(x & (x-1)); // Check if x is power of 2 (Safe
              for x=0)
long long lsb = x & -x; // Isolate lowest set bit (Update/Query
              in Fenwick Tree)
long long clear_lsb = x & (x-1); // Clear lowest set bit

// 2. Subelement Masks Enumeration
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {} // Iterate
all submasks of a state
void k_combinations(int n, int k) { // Gosper's Hack: Iterate
states with exactly k elements
    int mask = (1LL << k) - 1;
    while (mask < (1LL << n)) {
        int c = mask & -mask, r = mask + c;
        mask = (((mask ^ r) >> 2) / c) | r;
    }
}

// 3. Essential GCC Built-ins & __lg for CP
__builtin_popcountll(x) // Count set bits -> Size of subset in
                        Bitmask DP
__builtin_ctzll(x) // Count trailing zeros -> Quick index
of lowest set bit (x > 0)
__builtin_clzll(x) // Count leading zeros -> MSB position
analysis (x > 0)
__builtin_parityll(x) // True if x has an odd number of 1-
bits (Parity check)
__lg(x) // Fast Floor(log2(x)) -> Essential for
O(1) RMQ / Sparse Table query

// Next Power of 2 (Array padding / Segment Tree size allocation)
long long next_power_of_2(long long x) {
    if (x <= 1) return 1;

```

```

    return 1LL << (64 - __builtin_clzll(x - 1));
}

// Safe 64-bit Modular Multiplication (Prevents __int128 overhead)
long long safe_mod_mul(unsigned long long a, unsigned long long b,
    unsigned long long m) {
    long long res;
    if (__builtin_mul_overflow(a, b, &res)) {
        return (long long) (((unsigned __int128) a * b) % m);
    }
    return res % m;
}

```

6.3 Linear Basis (XOR Basis)

A vector space structure used to find the maximum possible XOR sum subset or check independence of integers in $O(B)$ per operation, where B is the bit-width.

```

struct XorBasis {
    static const int BITS = 60; // Up to 2^60
    long long basis[BITS];
    int sz = 0;

    XorBasis() {
        memset(basis, 0, sizeof(basis));
    }

    bool insert(long long mask) {
        for (int i = BITS - 1; i >= 0; i--) {
            if ((mask >> i) & 1) {
                if (!basis[i]) {
                    basis[i] = mask;
                    sz++;
                    return true;
                }
            }
        }
        return false; // Dependent element, not inserted
    }

    long long max_xor() {
        long long res = 0;
        for (int i = BITS - 1; i >= 0; i--) {
            if ((res ^ basis[i]) > res) {
                res ^= basis[i];
            }
        }
        return res;
    }
};

```

Initialize via `XorBasis b`. Use `b.insert(x)` to add an element to the vector space. Call `b.max_xor()` to compute the maximum possible XOR sum that can be formed using any subset of the inserted elements.

7 DPs

7.1 Aliens Trick (WQS Binary Search)

Optimizes DP states from $O(N \cdot K)$ to $O(N \log(\text{Max } C))$ for concave or convex cost functions where you must choose exactly K items. It works by introducing a penalty (λ) for each chosen item and binary searching for the optimal penalty.

```

int n, k;
vector<long long> a;

// Returns {min_cost, number_of_items_chosen} under the current
// penalty
pair<long long, int> check(long long lambda) {
    // dp[i] stores {best_cost, item_count}
    vector<pair<long long, int>> dp(n + 1, {1e18, 0});
    dp[0] = {0, 0};

    for (int i = 1; i <= n; i++) {
        // Base transition without creating a new group/item
        dp[i] = dp[i - 1];

        // speculative transition by starting a new item/group
        long long cost_transition = dp[i - 1].first + a[i] - lambda;

        if (cost_transition < dp[i].first) {
            dp[i] = {cost_transition, dp[i - 1].second + 1};
        }
    }
    return dp[n];
}

long long solve_alien() {
    long long l = 0, r = 1e12, ans = -1;

    while (l <= r) {
        long long mid = (l + r) >> 1;
        auto [cost, count] = check(mid);

        if (count >= k) {
            ans = cost + k * mid; // Reconstruct original cost by
            // removing penalty
            r = mid - 1;          // Adjust boundaries depending on
            // convexity
        } else {
            l = mid + 1;
        }
    }
    return ans;
}

```

Initialize your inputs, then call `solve_alien()` to run the optimization template over the penalty space coordinates.

7.2 Divide & Conquer Optimization

Reduces DP transitions from $O(N^2)$ to $O(N \log N)$ for relations of the form $dp[i][j] = \min_{k < j} \{dp[i-1][k] + cost(k, j)\}$. It requires the optimal transition index to satisfy monotonicity: $opt[i][j] \leq opt[i][j+1]$.

```

int n, g;
vector<long long> dp_before, dp_curr;

long long cost(int k, int j) {
    long long value = 0;
    return value;
}

void compute(int l, int r, int opt_l, int opt_r) {
    if (l > r) return;

    int mid = (l + r) >> 1;
    int best_k = -1;
    dp_curr[mid] = 1e18;

    for (int k = opt_l; k <= min(mid, opt_r); k++) {
        long long current_cost = dp_before[k] + cost(k, mid);

```

```

        if (current_cost < dp_curr[mid]) {
            dp_curr[mid] = current_cost;
            best_k = k;
        }

        compute(l, mid - 1, opt_l, best_k);
        compute(mid + 1, r, best_k, opt_r);
    }

    void solve_dp() {
        dp_before.assign(n + 1, 0);
        dp_curr.assign(n + 1, 0);

        for (int j = 1; j <= n; j++) {
            dp_before[j] = cost(0, j);
        }

        for (int i = 2; i <= g; i++) {
            compute(1, n, 0, n);
            dp_before = dp_curr;
        }
    }
}

```

7.3 Knuth Optimization

Reduces DP transitions from $O(N^3)$ to $O(N^2)$ for equations of the form $dp[i][j] = \min_{i < k < j} \{dp[i][k] + dp[k][j]\} + cost(i, j)$. It requires the cost function to satisfy the quadrangle inequality and monotonicity, which guarantees that the optimal transition point satisfies $opt[i][j-1] \leq opt[i][j] \leq opt[i+1][j]$.

```

int n;
vector<vector<long long>> dp;
vector<vector<int>> opt;

long long cost(int i, int j) {
    long long value = 0;
    return value;
}

void solve_dp() {
    dp.assign(n + 1, vector<long long>(n + 1, 0));
    opt.assign(n + 1, vector<int>(n + 1, 0));

    for (int i = 1; i <= n; i++) {
        opt[i][i] = i;

        for (int len = 2; len <= n; len++) {
            for (int i = 1; i + len - 1 <= n; i++) {
                int j = i + len - 1;
                dp[i][j] = 1e18;

                int opt_l = opt[i][j - 1];
                int opt_r = opt[i + 1][j];

                for (int k = opt_l; k <= min(j - 1, opt_r); k++) {
                    long long current_cost = dp[i][k] + dp[k + 1][j] +
                    cost(i, j);
                    if (current_cost < dp[i][j]) {
                        dp[i][j] = current_cost;
                        opt[i][j] = k;
                    }
                }
            }
        }
    }
}

```

8 Geometry

8.1 Convex Hull (Andrew's Monotone Chain)

Computes the convex hull of a set of 2D integer points in $O(N \log N)$ time. It sorts points lexicographically and builds the upper and lower hulls separately.

```

struct Point {
    long long x, y;
    bool operator<(const Point& o) const {
        if (x != o.x) return x < o.x;
        return y < o.y;
    }
};

long long cross(Point a, Point b, Point c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}

vector<Point> convex_hull(vector<Point>& pts) {
    int n = pts.size(), k = 0;
    if (n <= 3) return pts;
    vector<Point> hull(2 * n);
    sort(pts.begin(), pts.end());

    // Build lower hull
    for (int i = 0; i < n; ++i) {
        while (k >= 2 && cross(hull[k - 2], hull[k - 1], pts[i]) <= 0) k--;
        hull[k++] = pts[i];
    }

    // Build upper hull
    for (int i = n - 2; i >= 0; i--) {
        while (k >= t && cross(hull[k - 2], hull[k - 1], pts[i]) <= 0) k--;
        hull[k++] = pts[i];
    }

    hull.resize(k - 1);
    return hull;
}

```

Pass a `vector<Point>` to `convex_hull(pts)`. The returned points are ordered counter-clockwise. To allow collinear points on the hull edge, change `<= 0` to `< 0` inside both loops.

8.2 Geometry Templates (Integer & Double)

Complete 2D geometry templates. The integer version guarantees 100

```

// 1. INTEGER GEOMETRY TEMPLATE
struct PointInt {
    long long x, y;
    PointInt(long long x = 0, long long y = 0) : x(x), y(y) {}
    PointInt operator+(const PointInt& o) const { return {x + o.x, y + o.y}; }
    PointInt operator-(const PointInt& o) const { return {x - o.x, y - o.y}; }
};

long long dot(PointInt a, PointInt b) { return a.x * b.x + a.y * b.y; }
long long cross(PointInt a, PointInt b) { return a.x * b.y - a.y * b.x; }
long long dist2(PointInt a, PointInt b) { return dot(a - b, a - b); }

```

```
int ccw(PointInt a, PointInt b, PointInt c) {
    long long cr = cross(b - a, c - a);
    if (cr == 0) return 0;
    return (cr > 0) ? 1 : -1;
}

bool on_segment(PointInt p, PointInt a, PointInt b) {
    return ccw(a, b, p) == 0 && dot(a - p, b - p) <= 0;
}

bool intersect(PointInt a, PointInt b, PointInt c, PointInt d) {
    if (on_segment(c, a, b) || on_segment(d, a, b) || on_segment(a,
        c, d) || on_segment(b, c, d)) return true;
    return ccw(a, b, c) * ccw(a, b, d) < 0 && ccw(c, d, a) * ccw(c,
        d, b) < 0;
}
```

```
// 2. DOUBLE GEOMETRY TEMPLATE
const double EPS = 1e-9;
```

```
struct PointDbl {
    double x, y;
    PointDbl(double x = 0, double y = 0) : x(x), y(y) {}
    PointDbl operator+(const PointDbl& o) const { return {x + o.x,
        y + o.y}; }
    PointDbl operator-(const PointDbl& o) const { return {x - o.x,
        y - o.y}; }
    PointDbl operator*(double c) const { return {x * c, y * c}; }
    PointDbl operator/(double c) const { return {x / c, y / c}; }
};

double dot(PointDbl a, PointDbl b) { return a.x * b.x + a.y * b.y;
}
double cross(PointDbl a, PointDbl b) { return a.x * b.y - a.y * b.x;
}
double dist2(PointDbl a, PointDbl b) { return dot(a - b, a - b); }
double dist(PointDbl a, PointDbl b) { return sqrt(dist2(a, b)); }
```

```
int ccw(PointDbl a, PointDbl b, PointDbl c) {
    double cr = cross(b - a, c - a);
    if (abs(cr) < EPS) return 0;
    return (cr > 0) ? 1 : -1;
}

bool on_segment(PointDbl p, PointDbl a, PointDbl b) {
    return ccw(a, b, p) == 0 && dot(a - p, b - p) <= EPS;
}

bool intersect(PointDbl a, PointDbl b, PointDbl c, PointDbl d) {
    if (on_segment(c, a, b) || on_segment(d, a, b) || on_segment(a,
        c, d) || on_segment(b, c, d)) return true;
    return ccw(a, b, c) * ccw(a, b, d) < 0 && ccw(c, d, a) * ccw(c,
        d, b) < 0;
}
```

```
PointDbl line_intersection(PointDbl a, PointDbl b, PointDbl c,
    PointDbl d) {
    double cp = cross(b - a, d - c);
    assert(abs(cp) > EPS);
    return a + (b - a) * (cross(c - a, d - c) / cp);
}
```

8.3 Incenter & Circumcenter of a Triangle

Computes the incenter (intersection of angle bisectors) and circumcenter (intersection of perpendicular bisectors) of a triangle defined by three 2D points.

```
struct Point {
    double x, y;
```

```
Point(double x = 0, double y = 0) : x(x), y(y) {}
Point operator+(const Point& o) const { return {x + o.x, y + o.
    y}; }
Point operator-(const Point& o) const { return {x - o.x, y - o.
    y}; }
Point operator*(double c) const { return {x * c, y * c}; }
Point operator/(double c) const { return {x / c, y / c}; }
};

double dist(Point a, Point b) {
    return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.
        y));
}

// 1. Computes the Incenter of Triangle ABC
Point get_incenter(Point A, Point B, Point C) {
    double a = dist(B, C);
    double b = dist(A, C);
    double c = dist(A, B);
    double perimeter = a + b + c;
    assert(perimeter > 1e-9);

    return (A * a + B * b + C * c) / perimeter;
}
```

```
// 2. Computes the Circumcenter of Triangle ABC
Point get_circumcenter(Point A, Point B, Point C) {
    double d = 2 * (A.x * (B.y - C.y) + B.x * (C.y - A.y) +
        A.y - B.y));
    assert(abs(d) > 1e-9); // Ensure points are not collinear

    double ux = ((A.x * A.x + A.y * A.y) * (B.y - C.y) +
        (B.x * B.x + B.y * B.y) * (C.y - A.y) +
        (C.x * C.x + C.y * C.y) * (A.y - B.y)) / d;

    double uy = ((A.x * A.x + A.y * A.y) * (C.x - B.x) +
        (B.x * B.x + B.y * B.y) * (A.x - C.x) +
        (C.x * C.x + C.y * C.y) * (B.x - A.x)) / d;

    return {ux, uy};
}
```

Call `get_incenter(A, B, C)` OR `get_circumcenter(A, B, C)` by passing three Point structs. Ensure that the three coordinates do not lie on a straight line to prevent division by zero errors.

8.4 Li Chao Tree

Maintains a set of lines or line segments to query the maximum or minimum value at any integer coordinate x within a fixed range $[L, R]$ in $O(\log(R - L))$.

```
const long long INF = 1e18;

struct Line {
    long long k, m;
    long long operator()(long long x) { return k * x + m; }
};

struct LiChaoTree {
    int n;
    vector<Line> tree;
    LiChaoTree(int n) : n(n), tree(4 * n, {0, -INF}) {} // Set to
        {0, INF} for Minimum

    void insert(int node, int l, int r, Line newline) {
        int mid = (l + r) >> 1;
        // Change condition to '>' for Minimum
        bool bit_l = newline(l) > tree[node](l);
        bool bit_m = newline(mid) > tree[node](mid);
```

```
        if (bit_m) swap(tree[node], newline);
        if (1 == r) return;

        if (bit_l != bit_m) insert(node << 1, l, mid, newline);
        else insert(node << 1 | 1, mid + 1, r, newline);
    }

    long long query(int node, int l, int r, int x) {
        if (1 == r) return tree[node](x);
        int mid = (l + r) >> 1;
        long long res = tree[node](x);
        if (x <= mid) res = max(res, query(node << 1, l, mid, x));
        // Use min() for Minimum
        else res = max(res, query(node << 1 | 1, mid + 1, r, x));
        // Use min() for Minimum
        return res;
    }
};
```

This default implementation computes the **MAXIMUM** over the coordinate range $[1, N]$.

- **To compute Minimum:** Change the initialization line dummy to `{0, INF}`, replace both line comparisons with `>` to `<`, and substitute overall node merging functions from `max()` to `min()`.

8.5 LineContainer (CHT)

Maintains a dynamic set of lines $y = kx + m$ to query the maximum or minimum value at any x in $O(\log N)$ using a variant of `std::set`.

```
struct Line {
    mutable long long k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(long long x) const { return p < x; }
};

struct LineContainer : set<Line, less<>> {
    static const long long INF = LLONG_MAX;

    long long div(long long a, long long b) {
        return a / b - ((a ^ b) < 0 && a % b);
    }

    bool isect(iterator x, iterator y) {
        if (y == end()) { x->p = INF; return false; }
        if (x->k == y->k) x->p = x->m > y->m ? INF : -INF;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }

    void add(long long k, long long m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) { isect(x, y = erase(y))
        ; }
        while ((y = x) != begin() && (--x)->p >= y->p) isect(x,
            erase(y));
    }

    long long query(long long x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};
```

This structure natively computes the **MAXIMUM** value.

- **To find Maximum:** Call `add(k, m)` and query directly via `query(x)`.

- **To find Minimum:** Insert lines as `add(-k, -m)`. When querying, negate the output: `-query(x)`.

8.6 Polygon Area (Shoelace Formula)

Computes the area of a polygon in $O(N)$ given its vertices ordered sequentially along the perimeter.

```
struct Point {
    long long x, y;
};

long long cross(Point a, Point b) {
    return a.x * b.y - a.y * b.x;
}

double polygon_area(const vector<Point>& pts) {
    long long area2 = 0;
    int n = pts.size();
    for (int i = 0; i < n; i++) {
        int next_i = (i + 1 == n) ? 0 : i + 1;
        area2 += cross(pts[i], pts[next_i]);
    }
    return abs(area2) / 2.0;
}
```

Pass vertices sequentially (either clockwise or counter-clockwise). The function returns the true floating-point area by evaluating the absolute value of the total signed cross-product sum divided by two.

8.7 Smallest Enclosing Circle (Welzl's Algorithm)

Finds the unique minimum bounding circle enclosing a set of 2D points in $O(N)$ expected time using a randomized incremental approach.

```
#include <random>
#include <algorithm>
#include <cmath>

struct Point {
    double x, y;
};

struct Circle {
    Point center;
    double r;
};

double dist(Point a, Point b) {
    return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y));
}

bool is_inside(Point p, Circle c) {
    return dist(p, c.center) <= c.r + 1e-9;
}

Point get_circle_center(double bx, double by, double cx, double cy) {
    double B = bx * bx + by * by;
    double C = cx * cx + cy * cy;
    double D = bx * cy - by * cx;
```

```
    return {(cy * B - by * C) / (2 * D), (bx * C - cx * B) / (2 * D)};
}

Circle circle_from_3_points(Point A, Point B, Point C) {
    Point center = get_circle_center(B.x - A.x, B.y - A.y, C.x - A.x, C.y - A.y);
    center.x += A.x; center.y += A.y;
    return {center, dist(center, A)};
}

Circle circle_from_2_points(Point A, Point B) {
    Point center = {(A.x + B.x) / 2.0, (A.y + B.y) / 2.0};
    return {center, dist(A, B) / 2.0};
}

Circle welzl(vector<Point>& pts, vector<Point> r, int n) {
    if (n == 0 || r.size() == 3) {
        if (r.empty()) return {{0, 0}, 0};
        if (r.size() == 1) return {r[0], 0};
        if (r.size() == 2) return circle_from_2_points(r[0], r[1]);
        return circle_from_3_points(r[0], r[1], r[2]);
    }

    Point p = pts[n - 1];
    Circle c = welzl(pts, r, n - 1);

    if (is_inside(p, c)) return c;

    r.push_back(p);
    return welzl(pts, r, n - 1);
}

Circle get_smallest_enclosing_circle(vector<Point> pts) {
    mt19937 rng(1337);
    shuffle(pts.begin(), pts.end(), rng);
    return welzl(pts, {}, pts.size());
}
```

Call `get_smallest_enclosing_circle(pts)` to compute the optimal bounding box structure. Shuffling the vector inputs beforehand is mandatory to preserve the linear average time complexity bounds.

9 Misc

9.1 Custom Hash for hashmap & hashset

A secure custom hash structure to prevent $O(N^2)$ worst-case time complexity attacks (anti-hash-kill) caused by malicious test case inputs.

```
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM = chrono::steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
};
```

To use this structure, pass `custom_hash` as the third template argument when defining your hash container:

```
unordered_map<long long, int, custom_hash> safe_map;
unordered_set<long long, custom_hash> safe_set;
```

9.2 Hill Climbing (Heuristics)

A local search meta-heuristic that continuously moves in the direction of increasing/decreasing value (ascending the hill) to find an optimal state. Unlike Simulated Annealing, it is purely greedy and can get trapped in local optima.

Core Methodology:

1. **Generate Initial State:** Construct a valid random configuration and calculate its objective cost function score.
2. **Evaluate Neighbors:** Generate small mutations or modifications to the active parameters (e.g., swapping two elements).
3. **Greedy Step:** Compare the mutated score with the original. If the mutant state improves the objective criteria, instantly accept it as the new baseline. Otherwise, discard it immediately.
4. **Termination:** Repeat until no neighboring modifications yield any further improvements (local peak reached), or the allocated execution time limit expires.

Strategies to Escape Local Optima (Random Restart): Because pure Hill Climbing stops at the very first local peak it encounters, combine it with a **Random Restart** strategy inside competitive programming constraints:

- Enclose the entire hill climbing block within an external loop or time-guard window
(`while (clock() < 0.95 * CLOCKS_PER_SEC)`).
- Whenever a state becomes completely stagnant (no mutations improve the score anymore), save its peak metrics to a global maximum tracker.
- Fully randomize the current state configuration back to zero and launch a brand new hill climbing ascent from that arbitrary coordinate.

9.3 Simulated Annealing (Heuristics)

A randomized optimization technique used to find global minima/maxima for NP-hard problems (e.g., TSP) by slowly lowering a temperature T parameter to escape local optima.

```
mt19937_64 sa_rng(1337);
inline double get_rand() { return (double)sa_rng() / mt19937_64::
    max(); }

struct State {
    long long cost;
    void init() { /* Construct initial valid state and calculate
        cost */ }
    void mutate() { /* Apply a small random perturbation to state
        */ }
    void revert() { /* Revert the mutation if rejected */ }
};

void simulated_annealing() {
    State current; current.init();
    State best = current;

    double T = 1e5;          // Initial temperature
    double T_min = 1e-3;     // Minimum cut-off temperature
    double alpha = 0.9995;   // Cooling schedule factor

    auto start_time = chrono::steady_clock::now();

    while (T > T_min) {
        // Time-limit safety guard (e.g., break at 0.95 seconds to
        prevent TLE)
        auto current_time = chrono::steady_clock::now();
        if (chrono::duration<double>(current_time - start_time).
            count() > 0.95) break;

        long long old_cost = current.cost;
        current.mutate();
        long long new_cost = current.cost;

        long long diff = new_cost - old_cost; // Assume
        minimization problem

        if (diff < 0 || exp(-diff / T) > get_rand()) {
            if (new_cost < best.cost) best = current; // Accept and
            update best
        } else {
            current.revert(); // Reject mutation
        }
        T *= alpha; // Cool down
    }
}
```

9.4 Policy-Based Data Structures (PBDS)

An extension structure in GCC that provides a high-performance Ordered Set, supporting index-based lookups (find_by_order) and order queries (order_of_key) in $O(\log N)$.

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

// Definition for Ordered Set
template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

// Definition for Ordered Multiset (allows duplicate keys)
template <typename T>
using ordered_multiset = tree<T, null_type, less_equal<T>,
    rb_tree_tag, tree_order_statistics_node_update>;
```

Basic operations and functions usage pattern:

```
ordered_set<int> st;
```

```
// 1. Insert elements normally
st.insert(1); st.insert(4); st.insert(8);

// 2. find_by_order(k): Returns an iterator to the k-th smallest
    element (0-indexed)
auto it = st.find_by_order(1); // Points to '4'
int val = *it;

// 3. order_of_key(x): Returns the number of elements strictly
    smaller than x
int cnt = st.order_of_key(5); // Returns 2 (elements 1 and 4 are
    smaller)

// 4. Erase element in ordered_multiset (MUST use iterator to avoid
    removing all duplicates)
ordered_multiset<int> mst;
mst.insert(4); mst.insert(4);
mst.erase(mst.find_by_order(mst.order_of_key(4))); // Safely erases
    only one '4'
```

9.5 Stress Tester

An automated, cross-platform stress testing harness using identical execution checking paths.

define int long long

```
// checker.cpp
int32_t main() {
    ifstream f_user("user.out"), f_brute("brute.out");
    string s_user, s_brute;
    while (f_user >> s_user) {
        if (!(f_brute >> s_brute) || s_user != s_brute) return 1;
    }
    if (f_brute >> s_brute) return 1;
    return 0;
}
```

```
// main.cpp
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count
    ());

int rand_range(int l, int r) {
    uniform_int_distribution<long long> dist(l, r);
    return dist(rng);
}

void gen_test() {
    ofstream out("test.in");
    out << rand_range(1, 100) << " " << rand_range(1, 100) << "\n";
}

int32_t main() {
    system("g++ -O3 user.cpp -o user");
    system("g++ -O3 brute.cpp -o brute");
    system("g++ -O3 checker.cpp -o checker");

    for (int t = 1; t <= 10000; ++t) {
        gen_test();
        system("./user < test.in > user.out");
        system("./brute < test.in > brute.out");
        if (system("./checker")) {
            cout << "WA on test " << t << "\n";
            return 0;
        }
    }
    cout << "All tests passed!\n";
    return 0;
}
```

10 Classic

10.1 Closest Pair of Points

Finds the closest pair of points in a 2D plane under Euclidean distance in $O(N \log N)$ time using a randomized / divide and conquer scanning approach.

```
struct Point {
    long long x, y, id;
    bool operator<(const Point& o) const {
        return x < o.x;
    }
};

bool compare_y(const Point& a, const Point& b) {
    return a.y < b.y;
}

double dist(Point a, Point b) {
    return sqrt((a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.
        y));
}

pair<Point, Point> closest_pair(vector<Point>& pts) {
    int n = pts.size();
    sort(pts.begin(), pts.end());

    double min_d = 1e18;
    pair<Point, Point> best_pair;
    set<Point, decltype(&compare_y)> active_set(&compare_y);

    int left = 0;
    for (int i = 0; i < n; i++) {
        while (pts[i].x - pts[left].x >= min_d) {
            active_set.erase(pts[left]);
            left++;
        }

        Point lower_bound_pt = {-1e18, (long long)(pts[i].y - min_d
            ), -1};
        auto it = active_set.lower_bound(lower_bound_pt);

        while (it != active_set.end() && it->y - pts[i].y < min_d)
        {
            double d = dist(pts[i], *it);
            if (d < min_d) {
                min_d = d;
                best_pair = {pts[i], *it};
            }
            it++;
        }
        active_set.insert(pts[i]);
    }
    return best_pair;
}
```

Pass a vector<Point> containing unique tracking IDs to closest_pair(pts). The function processes a sweep-line algorithm over the X-coordinate vector coordinates, filtering target subsets inside the std::set framework.

11 Formulas

11.1 Advanced Graph Conjectures & Theorems

A reference sheet of deep, specialized graph theory properties and bounds (e.g., Caccetta-Häggkvist, Chvátal's Toughness,

and Seymour's Neighborhoods) used for mathematical abstractions or finding hard structural edge cases.

The Caccetta-Häggkvist Conjecture (Girth & Degree Limits): Relates the minimum out-degree δ^+ of a directed graph (digraph) to the length of its shortest directed cycle (girth g).

- **Statement:** Any n -vertex directed graph with minimum out-degree $\delta^+ \geq r$ must contain a directed cycle of length at most $\lceil n/r \rceil$.
- **The Core $n/3$ Case:** For $r = n/3$, any digraph where every vertex has an out-degree $\geq n/3$ is guaranteed to contain a directed triangle (3-cycle).
- **Application Bound:** If a problem requires you to construct a directed graph without any small cycles of length $\leq L$, the maximum out-degree you can safely assign to any vertex is strictly bounded by:

$$\delta^+(u) \leq \frac{n-1}{L}$$

Häggkvist's Cycle Conjecture (Independent Edges): Guarantees when a set of matching edges can be entirely enclosed inside a single cycle based on degree constraints.

- **Definition (c_k -graph):** A graph of order n where every pair of non-adjacent vertices x, y satisfies $d(x) + d(y) \geq n + k$.
- **Statement:** Every set of independent (non-adjacent) edges in a c_1 -graph ($d(x) + d(y) \geq n + 1$) can be completely contained within a single cycle.

Chvátal's Toughness Conjecture (Hamiltonian Boundaries): Connects a graph's structural connectivity (toughness) directly to whether it contains a Hamilton cycle.

- **Toughness (t):** A graph G is t -tough if, for every subset of vertices S whose removal disconnects the graph, the number of resulting connected components $c(G \setminus S)$ satisfies:

$$t \leq \frac{|S|}{c(G \setminus S)}$$

- **Property:** Every Hamiltonian graph must be at least 1-tough (necessary condition).
- **Conjecture:** There exists a constant threshold t (originally hypothesized as 2, but proven to be larger) such that any t -tough graph is guaranteed to be Hamiltonian.

Seymour's Second Neighborhood Conjecture: Guarantees the existence of a vertex that spreads "wider" in its second hop than its first hop in directed graphs without 2-cycles.

- **Statement:** Every simple digraph without loops or 2-cycles (digons) contains at least one vertex v whose second neighborhood is at least as large as its first neighborhood:

$$|N^+(N^+(v)) \setminus N^+(v) \setminus \{v\}| \geq |N^+(v)|$$

Hadwiger's Conjecture (Minor vs. Coloring): A deep generalization of the Four Color Theorem mapping structural graph contractions to chromatic bounds.

- **Statement:** If a graph G cannot be colored with fewer than k colors ($\chi(G) \geq k$), then G must contain the complete graph K_k as a minor (can be formed by deleting vertices/edges and contracting edges).
- **Equivalences:** The case $k = 5$ is exactly equivalent to the Four Color Theorem for planar graphs.

11.2 Essential Mathematics Formulas

A concise compilation of counting sequences, identities, and structural counting theorems frequently encountered in competitive programming.

11.3 Combinatorics & Modular Precomputations

Precompute factorials and modular inverse factorials in $O(N)$ for $O(1)$ combination queries modulo prime M :

$$C_n^k = \frac{n!}{k!(n-k)!} \pmod{M}$$

$$\text{inv}[i] = -\lfloor M/i \rfloor \cdot \text{inv}[M \pmod{i}] \pmod{M}$$

11.4 Catalan Numbers

Counts binary search trees with n nodes, valid parenthesization pairs, and non-crossing polygon dissections.

$$C_0 = 1, \quad C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i} = \frac{1}{n+1} C_{2n}^n = C_{2n}^n - C_{2n}^{n-1}$$

The first few terms: 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862.

11.5 Derangements (Permutations with no fixed points)

Counts permutations of length n where no element i appears in its original i -th position.

$$D_1 = 0, \quad D_2 = 1, \quad D_n = (n-1)(D_{n-1} + D_{n-2}) = nD_{n-1} + (-1)^n$$

The first few terms: 0, 1, 2, 9, 44, 265, 1854, 14833, 133496.

11.6 Stirling Numbers of the Second Kind

$S(n, k)$ represents the number of ways to partition a set of n labeled elements into k unlabeled non-empty subsets.

$$S(n, k) = S(n-1, k-1) + k \cdot S(n-1, k)$$

Base conditions: $S(0, 0) = 1$, $S(n, 0) = S(0, n) = 0$. Number of ways to partition n labeled items into k labeled boxes is $k! \cdot S(n, k)$.

11.7 Lucas' Theorem

Computes $C_n^k \pmod{p}$ for a small prime p when $n, k \leq 10^{18}$. Express n and k in base p :

$$n = n_m p^m + n_{m-1} p^{m-1} + \dots + n_0, \quad k = k_m p^m + k_{m-1} p^{m-1} + \dots + k_0$$

$$C_n^k \equiv \prod_{i=0}^m C_{n_i}^{k_i} \pmod{p} \quad (\text{where } C_{n_i}^{k_i} = 0 \text{ if } n_i < k_i)$$

11.8 Graph Theory Counting

- **Cayley's Formula:** The number of unique labeled spanning trees in a complete graph K_n is n^{n-2} .
- **Handshaking Lemma:** $\sum \deg(v) = 2 \cdot |E|$. Total number of vertices with odd degree is always even.
- **Euler's Planar Formula:** For any connected planar graph: $V - E + F = 2$ (where F includes the outer infinite face).

11.9 Geometry Theorems

- **Pick's Theorem:** Calculates the area A of a polygon whose vertices all lie on an integer grid:

$$A = I + \frac{B}{2} - 1$$

where I is the number of interior grid points, and B is the number of boundary grid points.

- **Boundary Points on Segment:** The number of integer points strictly lying on the line segment connecting (x_1, y_1) and (x_2, y_2) including both endpoints is:

$$\gcd(|x_1 - x_2|, |y_1 - y_2|) + 1$$

11.10 Essential Graph and Number Theory Theorems

A cheat-sheet of critical theorems, invariants, and structural properties required for structural analysis and reduction tasks.
Graph Theory Invariants & Reductions:

- **Dilworth's Theorem:** In any finite partially ordered set (poset), the length of the maximum antichain (a subset of elements where no two are comparable) equals the minimum number of chains needed to cover the entire poset. (Reduces to Maximum Bipartite Matching via Konig's reduction).
- **Konig's Theorem:** In any bipartite graph, the number of edges in a maximum matching is equal to the number of vertices in a minimum vertex cover.
- **Vertex Cover vs. Independent Set:** For any graph $G = (V, E)$, a subset $S \subseteq V$ is a vertex cover if and only if its complement $V \setminus S$ is an independent set. Thus:
 $|Minimum\ Vertex\ Cover| + |Maximum\ Independent\ Set| = |V|$
- **Gallai's Identity:** In any graph without isolated vertices, the maximum matching size $\alpha'(G)$ and the minimum edge cover size $\beta'(G)$ satisfy:
$$\alpha'(G) + \beta'(G) = |V|$$
- **Matrix Tree Theorem (Kirchhoff's):** To find the total number of unique spanning trees in an undirected graph, construct the Laplacian matrix $L = D - A$, where D is the degree matrix and A is the adjacency matrix. Delete the i -th row and i -th column of L to form L_i . The number of spanning trees equals $\det(L_i)$.
- **Eulerian Path Conditions:** A connected undirected graph has an Eulerian cycle if and only if every vertex has an even degree. It has an Eulerian path if and only if exactly 0 or 2 vertices have an odd degree.

Number Theory Identities:

- **Euler's Theorem:** If $\gcd(a, m) = 1$, then $a^{\phi(m)} \equiv 1 \pmod{m}$.
- **Fermat's Little Theorem:** If p is a prime, then $a^p \equiv a \pmod{p}$, and $a^{p-1} \equiv 1 \pmod{p}$ when $\gcd(a, p) = 1$.
- **Bézout's Identity:** For any integers a and b with $d = \gcd(a, b)$, there exist integers x and y such that $ax + by = d$. The values can be fetched via Extended GCD.

- **Wilson's Theorem:** A positive integer $n > 1$ is a prime if and only if:

$$(n-1)! \equiv -1 \pmod{n}$$

- **Mobius Inversion Formula:** If $g(n) = \sum_{d|n} f(d)$ for every integer n , then $f(n) = \sum_{d|n} \mu(d)g(n/d)$, where $\mu(d)$ is the Mobius function.